

Manual de instalación de Storysphere



Eilyn Ortiz Guevara
Kebin Alejandro Manco Varela
Yerson Orejuela Arroyo

Servicio Nacional de Aprendizaje, Centro tecnológico del mobiliario
ADSO
04 de marzo de 2025

Tabla de contenido

Tabla de contenido	2
Información general del software	3
Descripción general	3
Propósito.....	4
Público objetivo	4
Tecnologías usadas	5
<i>Php con laravel</i>	<i>5</i>
<i>MYSQL.....</i>	<i>5</i>
<i>React.js con Vite.....</i>	<i>5</i>
Instalación y configuración.....	6
Requisitos previos	6
<i>Hardware:</i>	<i>6</i>
<i>Software.....</i>	<i>6</i>
Proceso de instalación paso a paso	7
<i>Visual Studio Code</i>	<i>7</i>
<i>React.js</i>	<i>15</i>
<i>Vite.....</i>	<i>25</i>
<i>Servidor apache</i>	<i>28</i>
<i>Composer.....</i>	<i>29</i>
Arquitectura del sistema	36
Estructura general del software	36
<i>Frontend – React.js con Vite.....</i>	<i>36</i>
<i>Backend – Laravel</i>	<i>37</i>

<i>Base de datos – MYSQL</i>	<i>37</i>
<i>Comunicación entre módulos.....</i>	<i>37</i>
Desarrollo y estructura del código	38
Organización del código (Carpetas y archivos)	38
Base de datos	38
Modelo de datos y diagramas entidad-relación	38
<i>Modelo de datos</i>	<i>38</i>
<i>Modelo entidad - relación.....</i>	<i>39</i>
Tablas principales y relaciones.....	39
Procedimientos almacenados y consultas SQL relevantes.....	40
Ejemplos de solicitudes y respuestas.....	44
Bibliografía.....	48
Instalaciones	48
Código fuente	48
Diagramas	48

Información general del software

Descripción general

Este manual proporciona las instrucciones necesarias para la instalación y configuración del software Storysphere en un entorno adecuado. Incluye los requisitos previos, la descarga de archivos, la instalación de dependencias y la verificación del correcto funcionamiento del sistema.

El objetivo es garantizar una instalación fluida y sin errores, permitiendo que el usuario configure el software de manera óptima para su uso.

Propósito

El propósito de este manual es facilitar la instalación y puesta en marcha de Storysphere, asegurando que los usuarios sigan un proceso claro y estructurado. Este documento está dirigido a desarrolladores y administradores de sistemas, proporcionando los pasos detallados y las mejores prácticas para evitar problemas técnicos durante la instalación.

Público objetivo

Este manual está dirigido a desarrolladores, administradores de sistemas y cualquier persona encargada de la instalación y configuración de la página. Se asume que los usuarios tienen conocimientos básicos en desarrollo web, bases de datos y administración de servidores. Los principales perfiles que pueden beneficiarse de este manual incluyen:

- Desarrolladores Backend que necesiten instalar y configurar la API en Laravel.
- Desarrolladores Frontend que trabajen con React.js y requieran conectar la interfaz con el backend.
- Administradores de Bases de Datos encargados de gestionar SQL Server y su integración con la aplicación.
- Administradores de Sistemas que realicen la instalación en entornos de producción o servidores locales.

Tecnologías usadas

Php con laravel

PHP es un lenguaje de programación utilizado para el desarrollo web, permitiendo la integración con HTML y la ejecución de código en el servidor. Esto facilita la generación de contenido dinámico y la interacción con múltiples bases de datos.

Para este proyecto, se ha utilizado Laravel, un framework de PHP que ofrece una estructura organizada y herramientas avanzadas para el desarrollo rápido y eficiente de aplicaciones. Laravel ha sido empleado específicamente para la creación de una API, lo que facilita la comunicación entre el backend y el frontend, garantizando una arquitectura escalable y segura.

MYSQL

La base de datos utilizada en este proyecto es MYSQL Server, un sistema de gestión de bases de datos relacional. MYSQL Server proporciona un alto rendimiento, seguridad avanzada y opciones de escalabilidad, asegurando la integridad de los datos y optimizando las consultas. Entre sus características clave están:

- Alta disponibilidad y replicación para entornos de producción.
- Optimización de consultas mediante índices y procedimientos almacenados.
- Seguridad avanzada, incluyendo autenticación y cifrado de datos.

React.js con Vite

Para el desarrollo del frontend, se ha utilizado React.js con Vite, una biblioteca de JavaScript que permite la creación de interfaces de usuario dinámicas y eficientes. Vite se

encarga de optimizar la carga y compilación del código, mejorando la velocidad de desarrollo y la experiencia del usuario. Entre las ventajas de esta tecnología se encuentran:

- Renderizado eficiente gracias a la arquitectura basada en componentes.
- Desarrollo rápido con recarga en tiempo real y optimización del código.
- Integración fluida con estilos CSS dentro de JavaScript, facilitando la personalización de la interfaz.

Instalación y configuración

Requisitos previos

Hardware:

- Sistema Operativo: Windows 10 / Linux (Ubuntu 20.04 o superior) / macOS.
- Procesador: Intel Core i3 / AMD Ryzen 3 o superior.
- Memoria RAM: 2GB (se recomienda 8GB o más para un mejor rendimiento).
- Disco Duro: 10GB de espacio libre en el almacenamiento (preferiblemente SSD para mayor velocidad).
- Resolución de pantalla: 1366 x 768 píxeles (se recomienda Full HD o superior para mejor experiencia).
- Periféricos: Teclado y ratón (auriculares opcionales).

Software

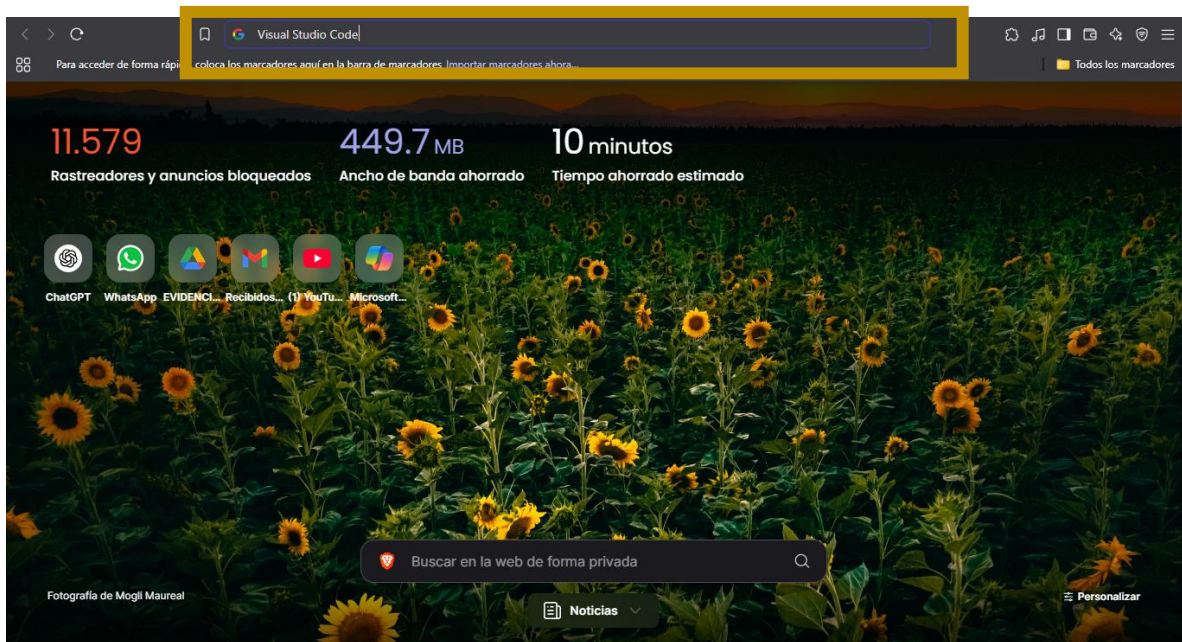
- Privilegios de administrador para la instalación y configuración del sistema.
- React.js versión 16 o superior.
- Npm (Node Package Manager) como gestor de paquetes para React.js.
- Git para la gestión del código y control de versiones.

- Visual Studio Code o cualquier otro editor de código compatible con Vite.
- MySQL como motor de base de datos para la gestión de información.

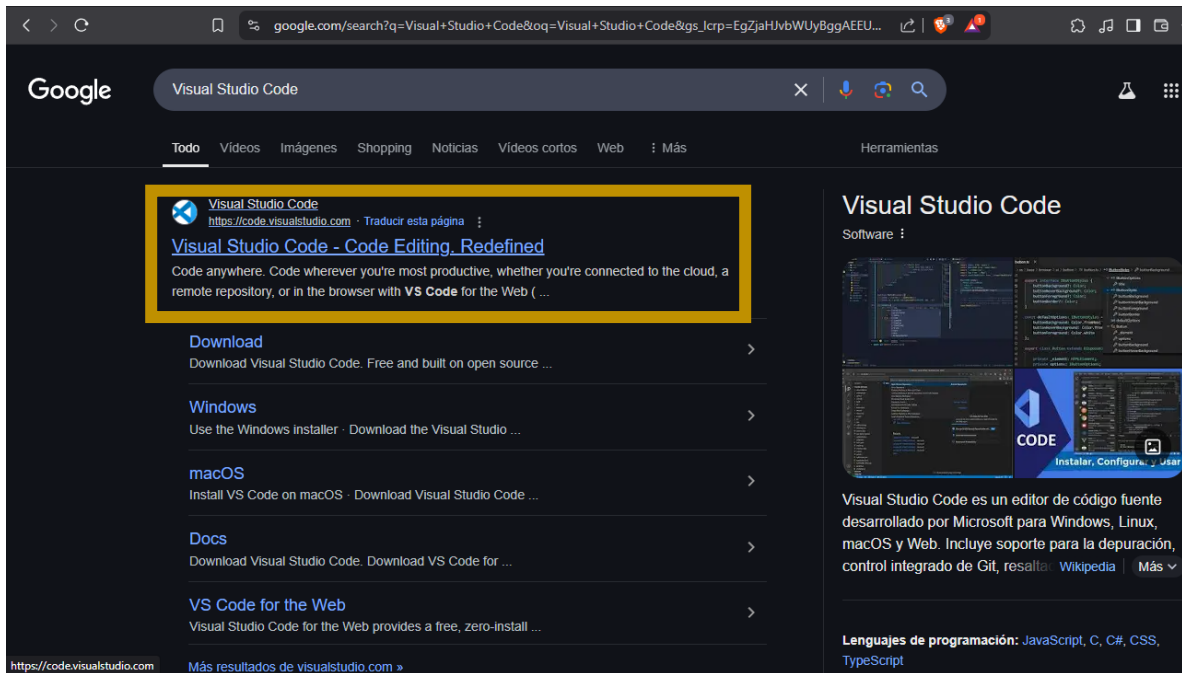
Proceso de instalación paso a paso

Visual Studio Code

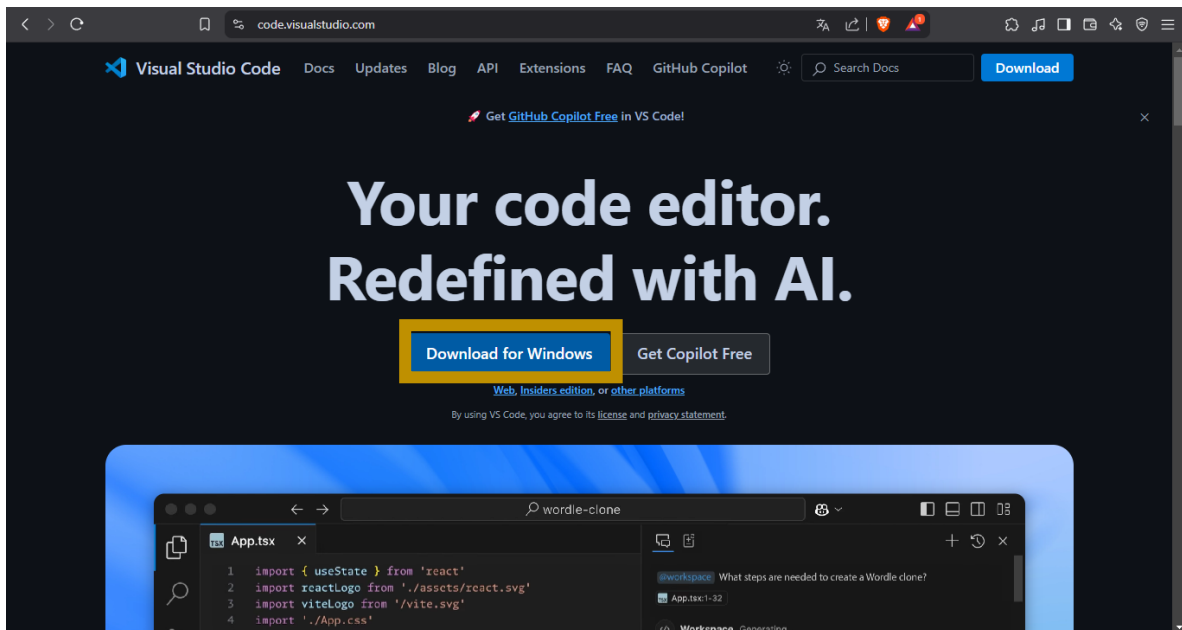
Para descargar, abra su navegador web favorito y en la barra de búsqueda escribe Visual Studio Code.



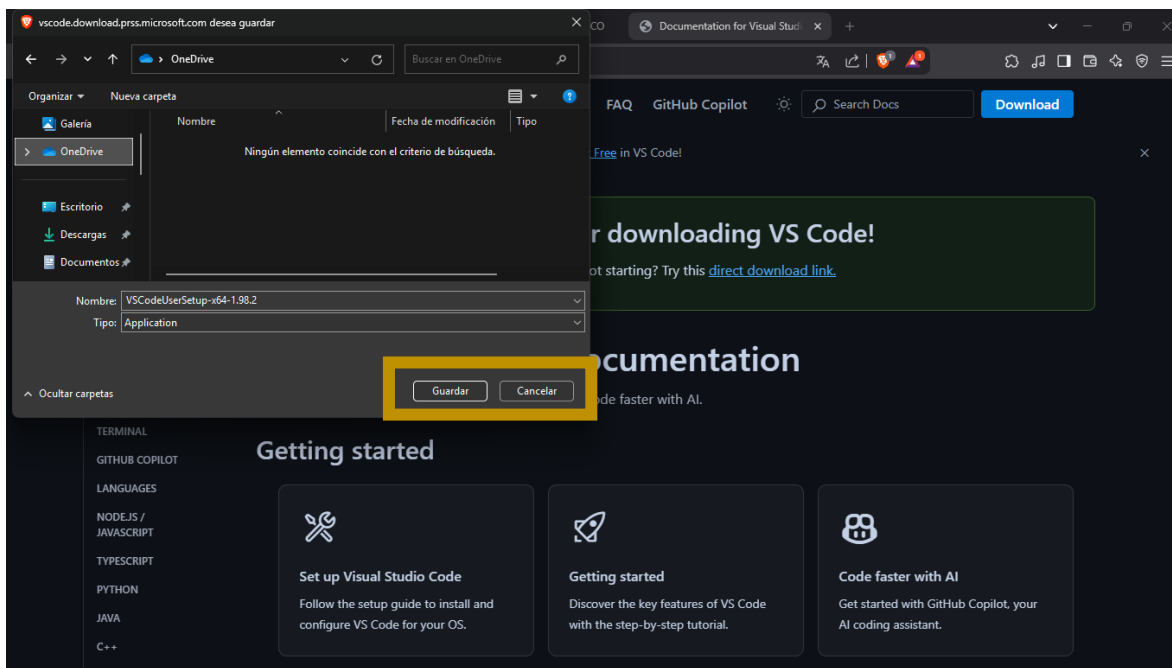
Dirigete al sitio oficial de Visual Studio Code y haga click



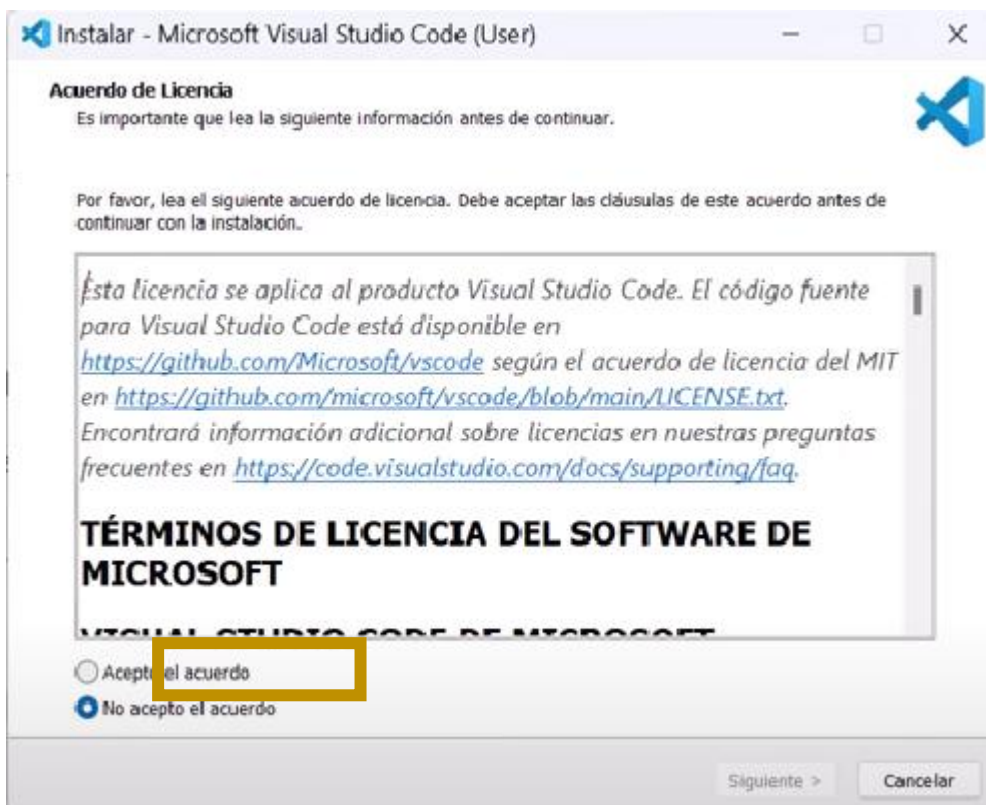
En esta ventana no será necesario seleccionar el sistema operativo ya que la misma página de Visual Studio Code lo identifica, siguiente haga click en descargar.



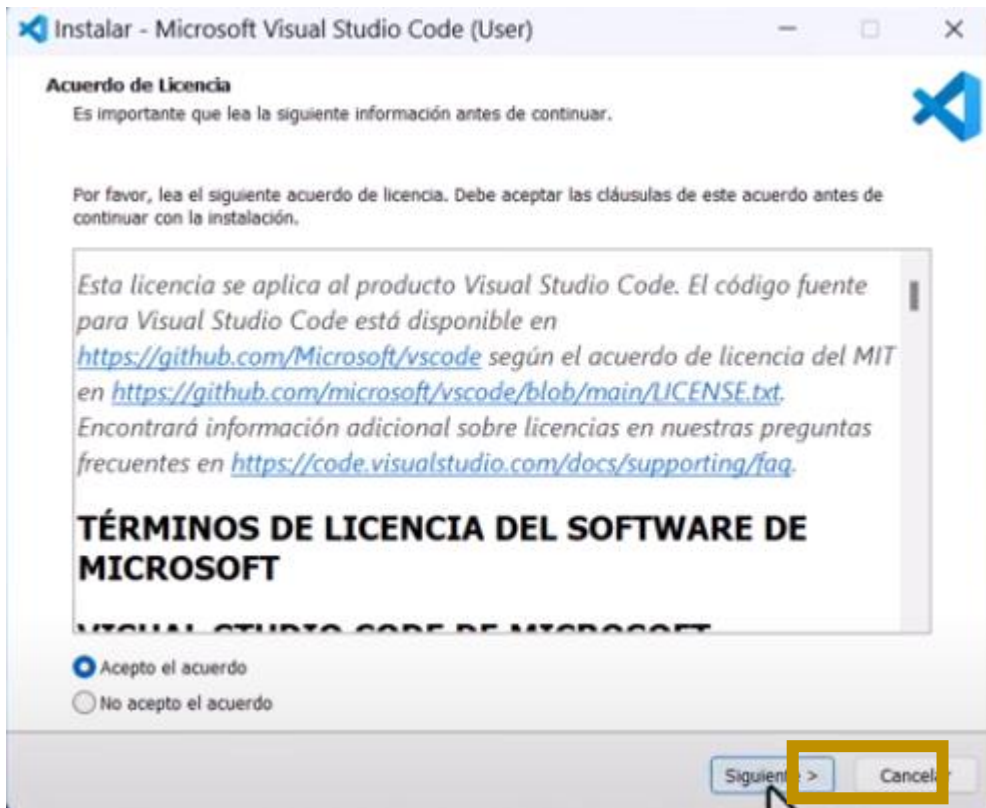
Para guardar el archivo ejecutable haga click en guardar



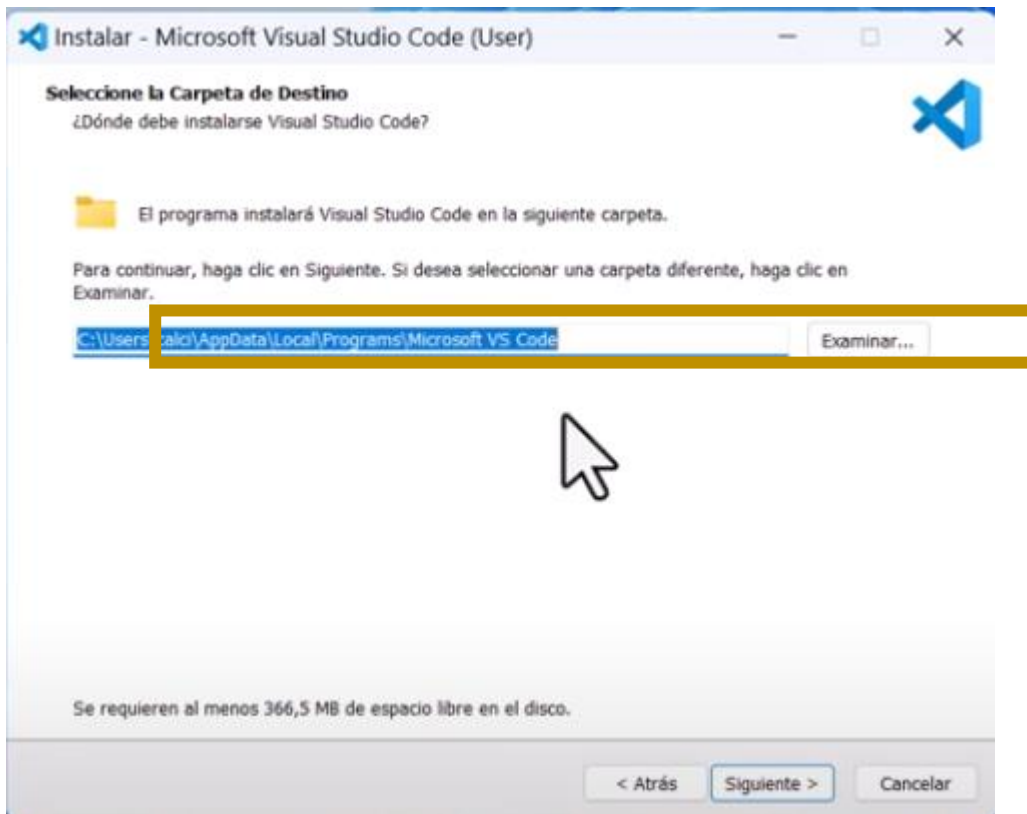
Automaticamente empezara a descargar, después que se complete el tiempo de descarga abra la aplicación.



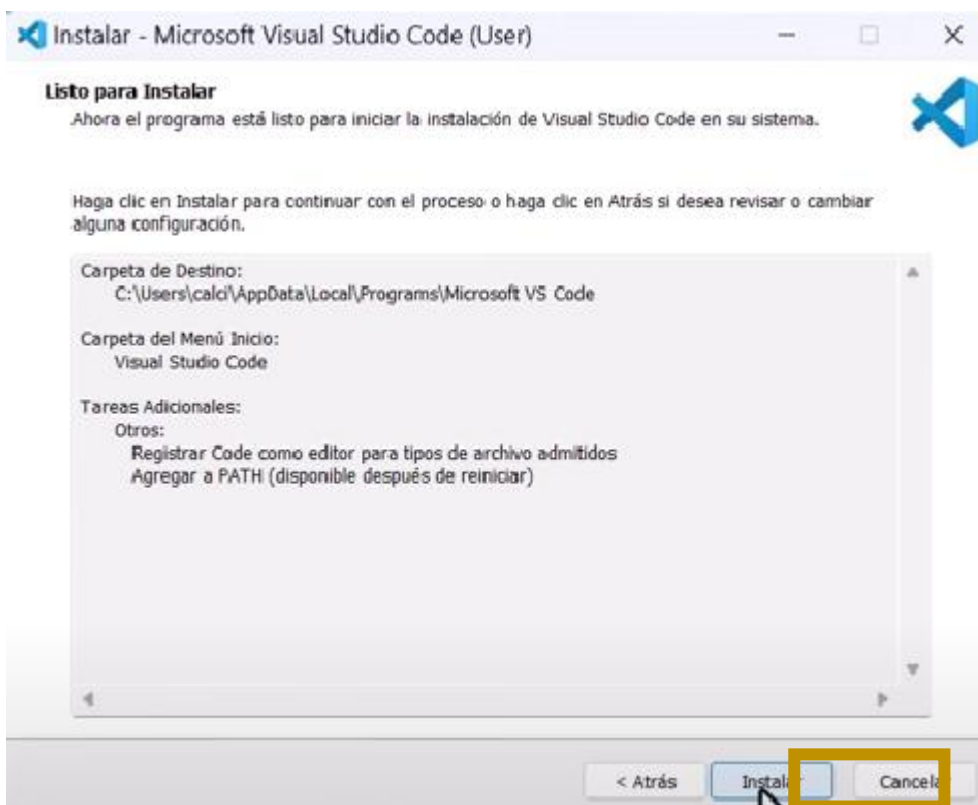
Haga click en la parte que dice “Acepto el acuerdo” aquí aceptarás la licencia.



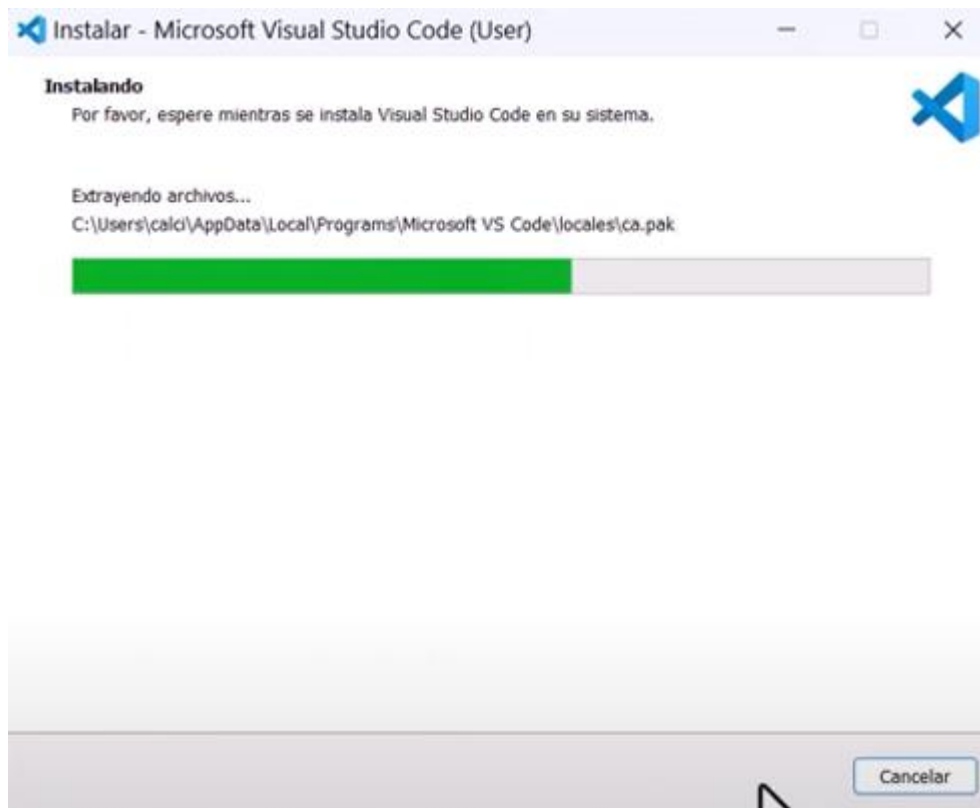
Cuando damos click es siguiente se instalara en la carpeta predeterminada



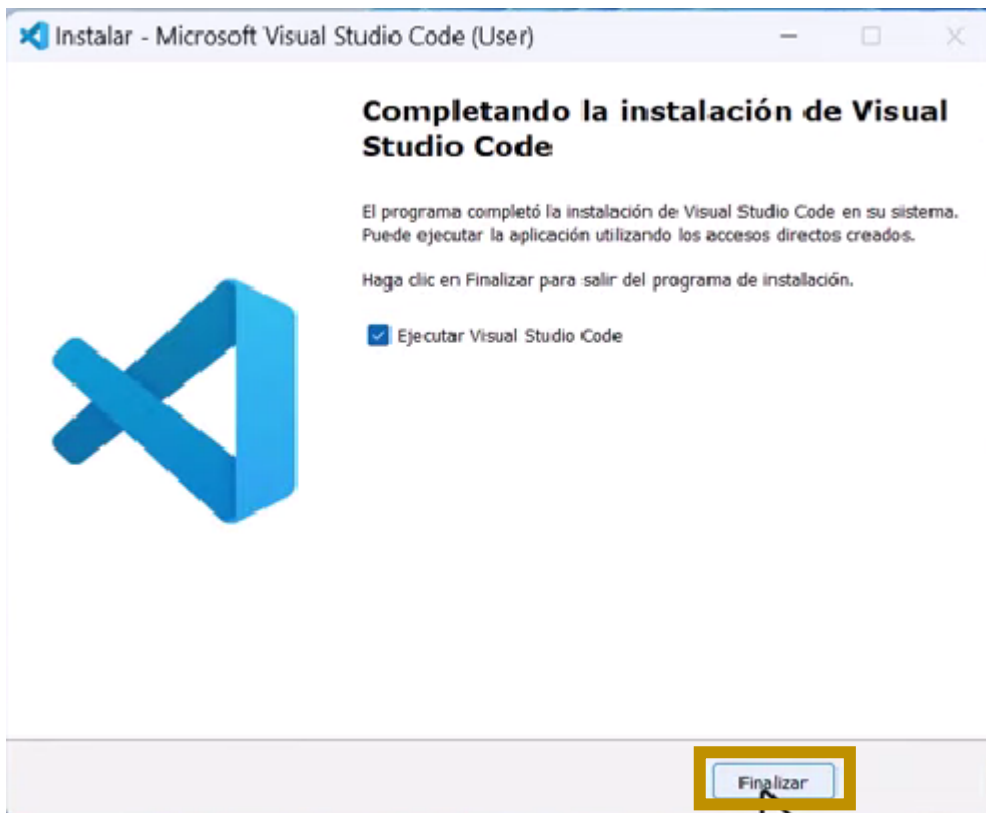
Si continuas click en siguiente tres veces y click en instalar



Espera que termine de instalarse

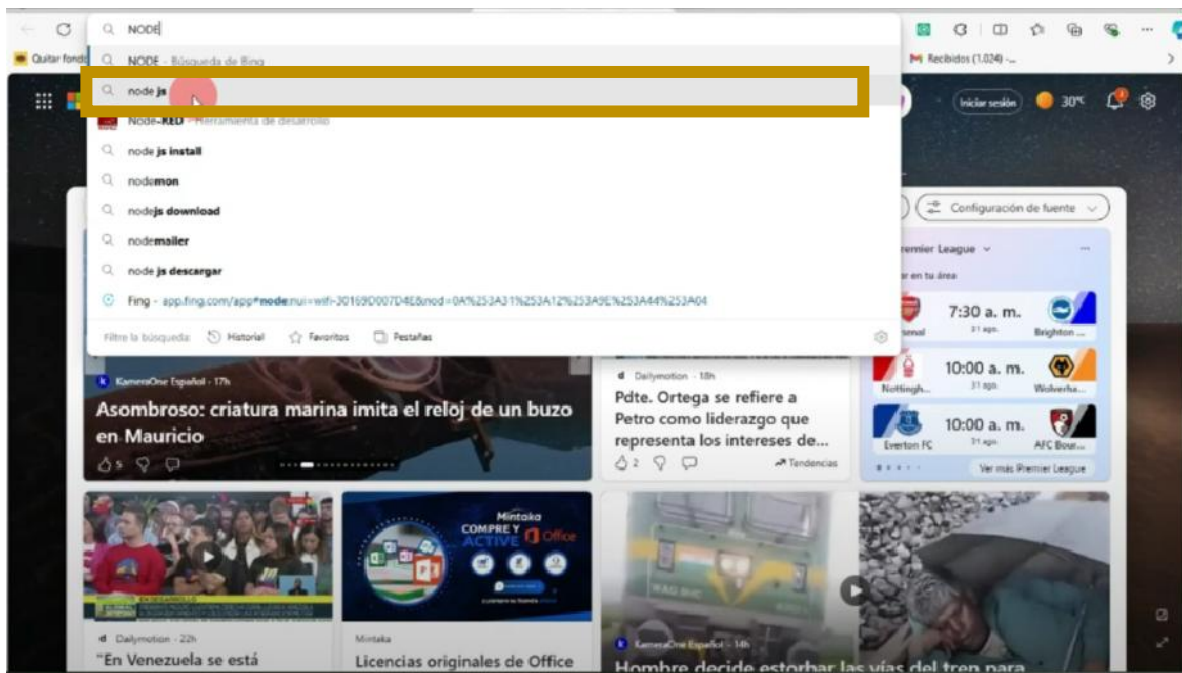


Una vez terminado click en “Finalizar” y finalmente se abrirá Visual Studio Code

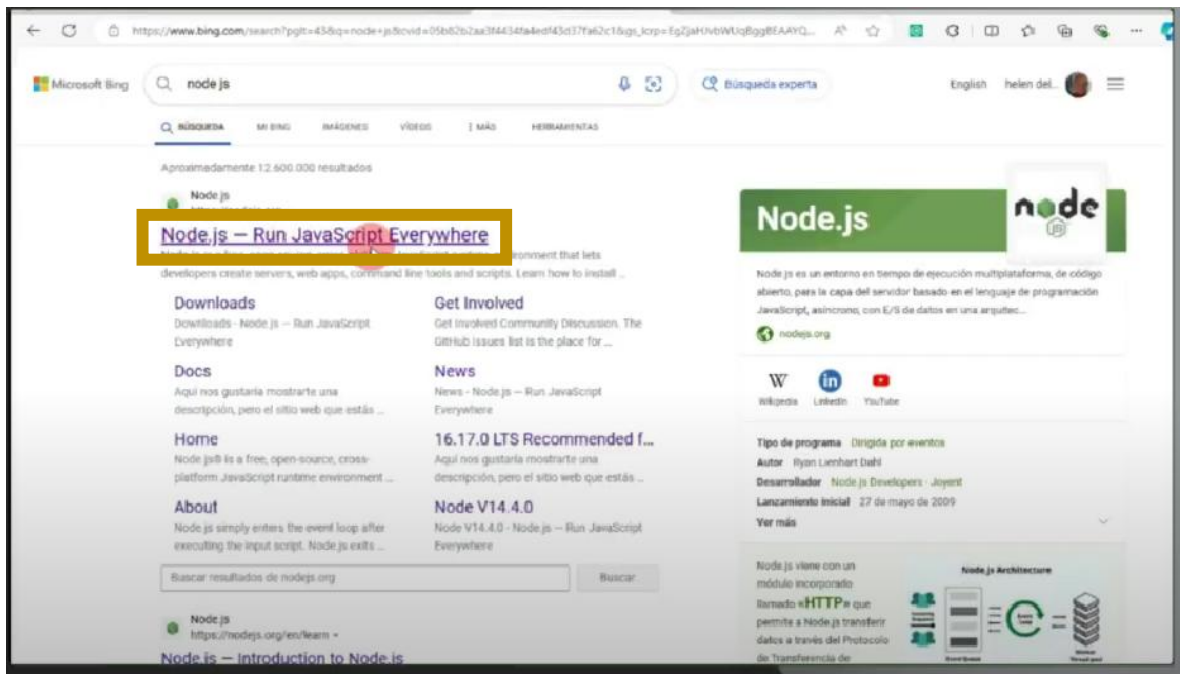


React.js

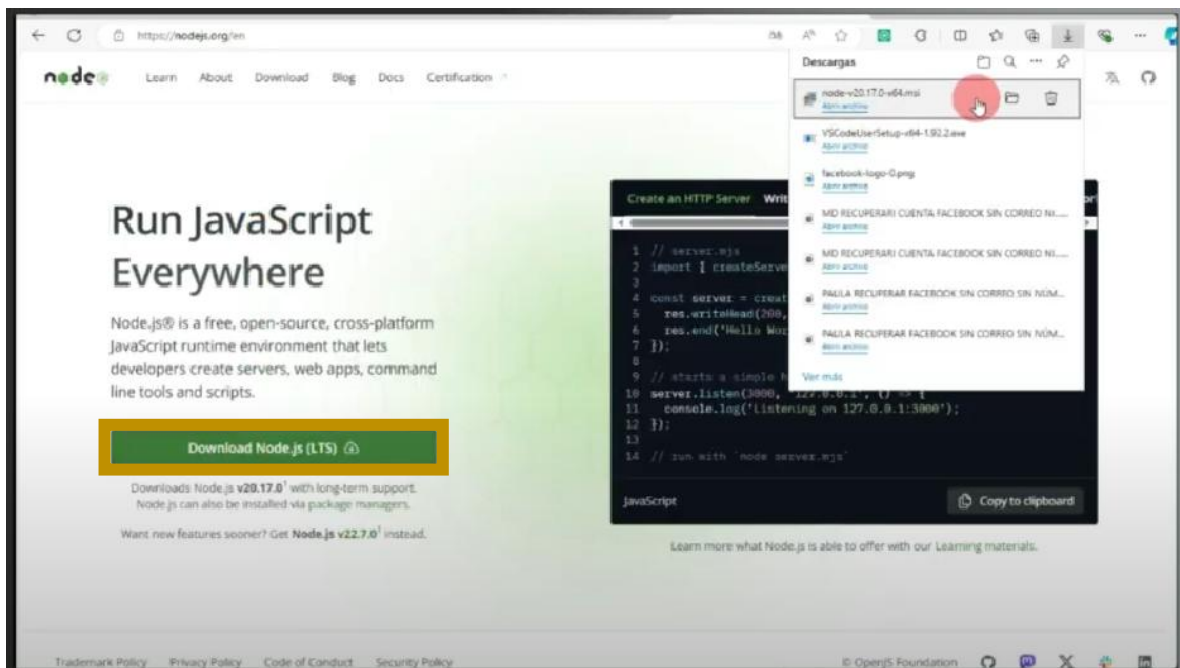
Para descargar, abra su navegador web favorito y en la barra de búsqueda escribe Node js.



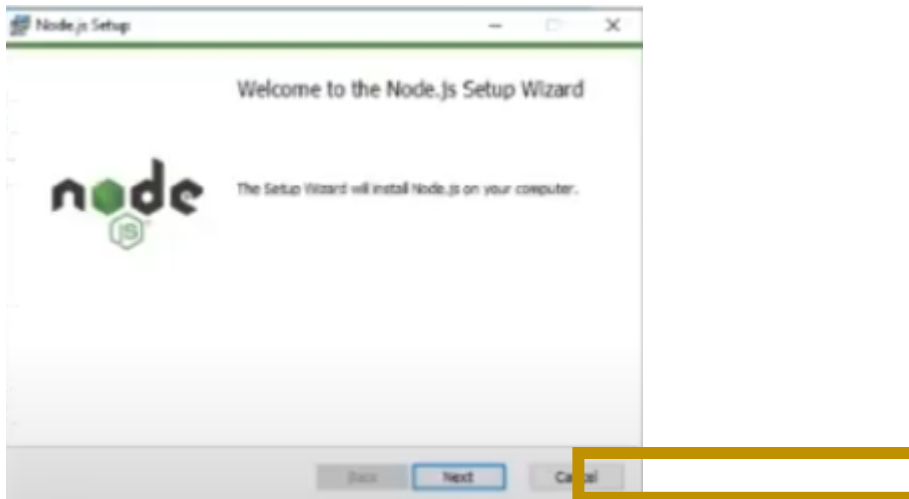
Dirigete al sitio oficial de Node js y haga click



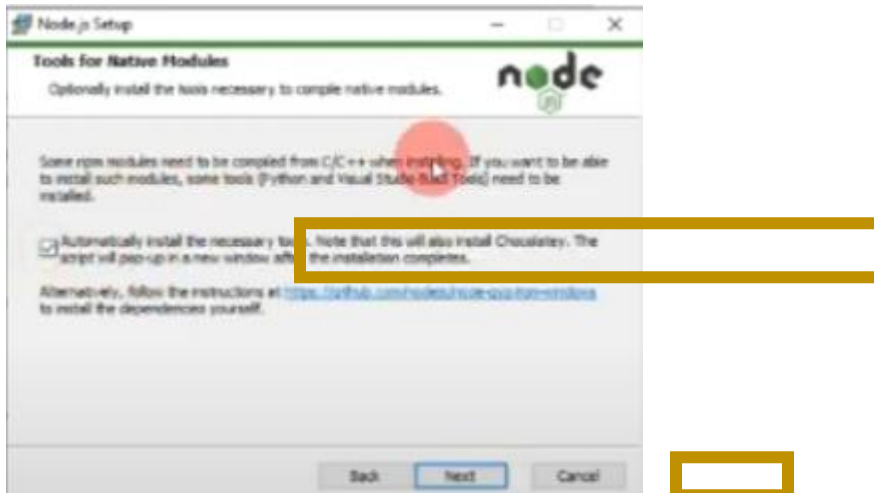
En esta ventana no será necesario seleccionar el sistema operativo ya que la misma página de Node js lo identifica, siguiente haga click en descargar.



Apenas descargue la carpeta, doble click en la misma para instalar.



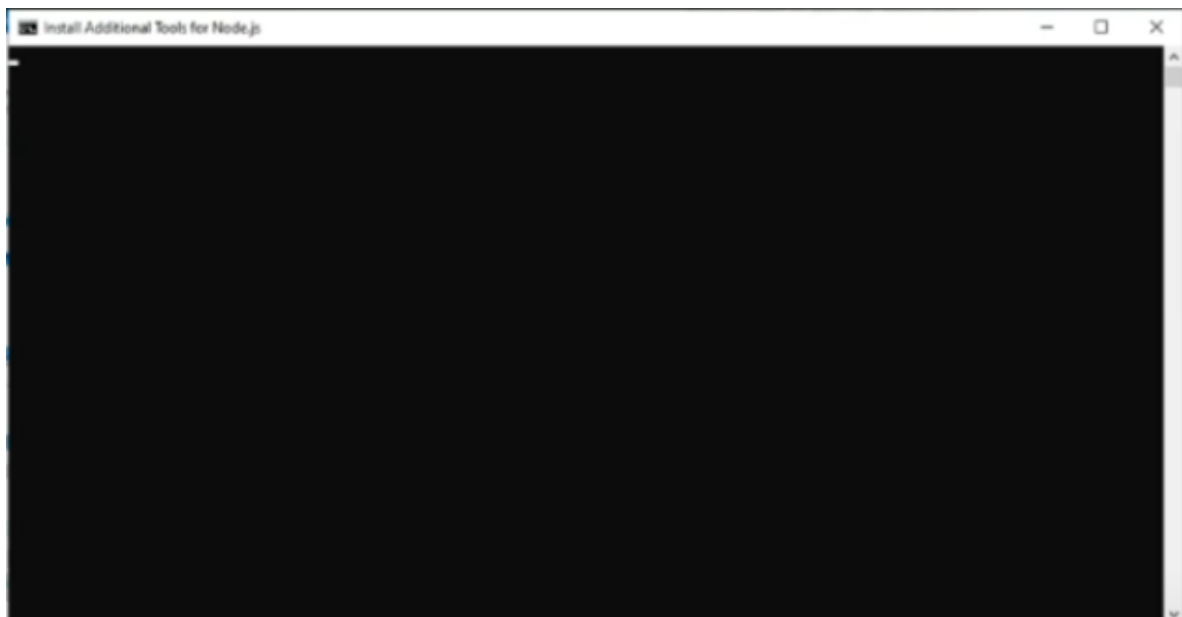
Aquí dale varias veces next hasta aparecer la siguiente pantalla.



Para avanzar click en donde muestra el recuadro dorado y siguiente next hasta que diga “Install”



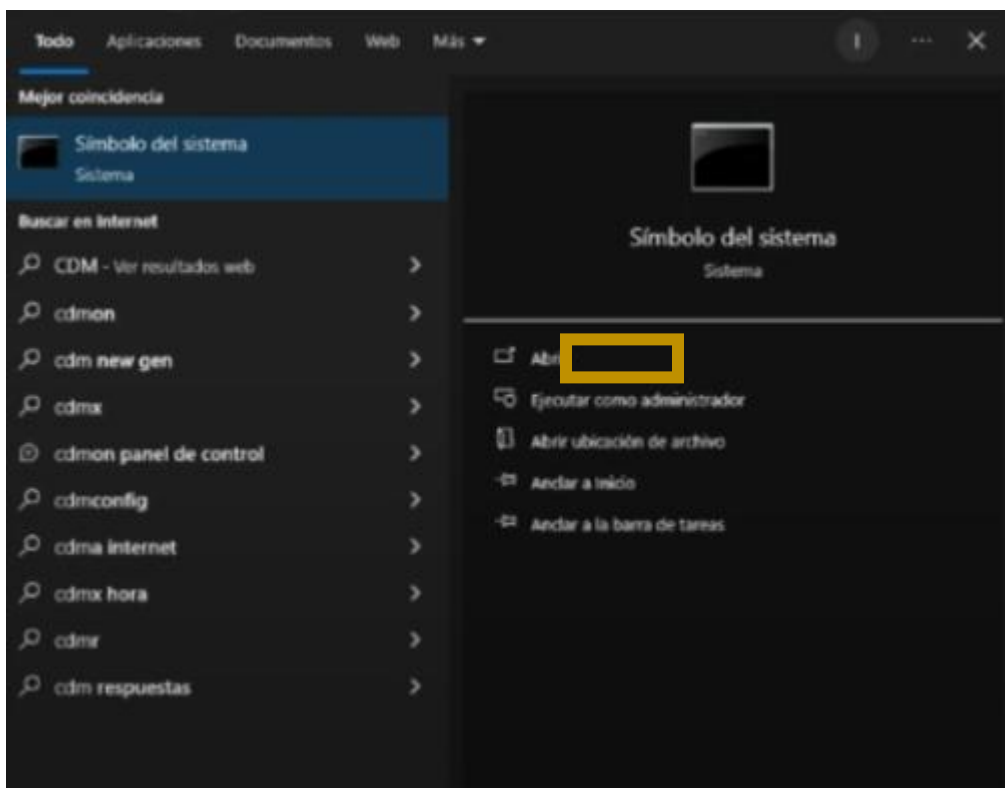
Seguidamente va a salir esta pantalla dale enter.



A continuación se muestra el Windows PowerShell que va a hacer un proceso.

```
Administración Windows PowerShell
Forcing web requests to allow TLS v1.2 (Required for requests to Chocolatey.org)
Getting latest version of the Chocolatey package for download.
Not using proxy.
Getting Chocolatey from https://community.chocolatey.org/api/v2/package/chocolatey/2.3.0.
Downloading https://community.chocolatey.org/api/v2/package/chocolatey/2.3.0 to C:\Users\admin\AppData\Local\Temp\choco\
chocolatey\chocoInstall\chocolatey.zip
Not using proxy.
Extracting C:\Users\admin\AppData\Local\Temp\chocolatey\chocoInstall\chocolatey.zip to C:\Users\admin\AppData\Local\Temp\
chocolatey\chocoInstall
Installing Chocolatey on the local machine
Creating ChocolateyInstall as an environment variable (targeting 'Machine')
Setting chocolateyInstall to 'C:\ProgramData\chocolatey'
WARNING: It's very likely you will need to close and reopen your shell
before you can use choco.
Restricting write permissions to Administrators
We are setting up the chocolatey package repository.
The packages themselves go to 'C:\ProgramData\chocolatey\lib'
(i.e. C:\ProgramData\chocolatey\lib\yourPackageName).
A shim file for the command line goes to 'C:\ProgramData\chocolatey\bin'
and points to an executable in 'C:\ProgramData\chocolatey\lib\yourPackageName'.
Creating chocolatey CLI folders if they do not already exist.
chocolatey.nupkg file not installed in lib.
Attempting to locate it from bootstrapper.
PATH environment variable does not have C:\ProgramData\chocolatey\bin in it. Adding...
ADVERTENCIA: Not setting tab completion: Profile file does not exist at
'C:\Users\admin\Documents\WindowsPowerShell\Microsoft.PowerShell_profile.ps1'.
chocolatey CLI (choco.exe) is now ready.
You can call choco from anywhere, command line or powershell by typing choco.
Run choco /? for a list of functions.
You may need to shut down and restart powershell and/or consoles
first prior to using choco.
Ensuring Chocolatey commands are on the path
Ensuring chocolatey.nupkg is in the lib folder
chocolatey v2.3.0
Upgrading the following packages:
python-cryptography-win32-urllib3-win32-vctools
By upgrading, you accept licenses for the packages.
Downloading package from source 'https://community.chocolatey.org/api/v2/'
Progress: Downloading chocolatey-compatibility.extension 1.0.0... 100%
chocolatey-compatibility.extension v1.0.0 [Approved]
chocolatey-compatibility.extension package files upgrade completed. Performing other installation steps.
```

Buscar en el logo de Windows “cmd” para que aparezca esto.



Al abrir esta consola deberas escribir lo siguiente para instalar el react

```
npm init react-app reactjs
Microsoft Windows [Versi3n 10.0.19045.4780]
(c) Microsoft Corporation. Todos los derechos reservados.
C:\Users\admin>cd c:\reactjs
El sistema no puede encontrar la ruta especificada.
C:\Users\admin>npm init react-app reactjs
```

Dile que si con la letra “Y”, das enter y esperas hasta que te salga el siguiente mensaje.

```
Símbolo del sistema
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.
Success! Created reactjs at C:\Users\admin\reactjs
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

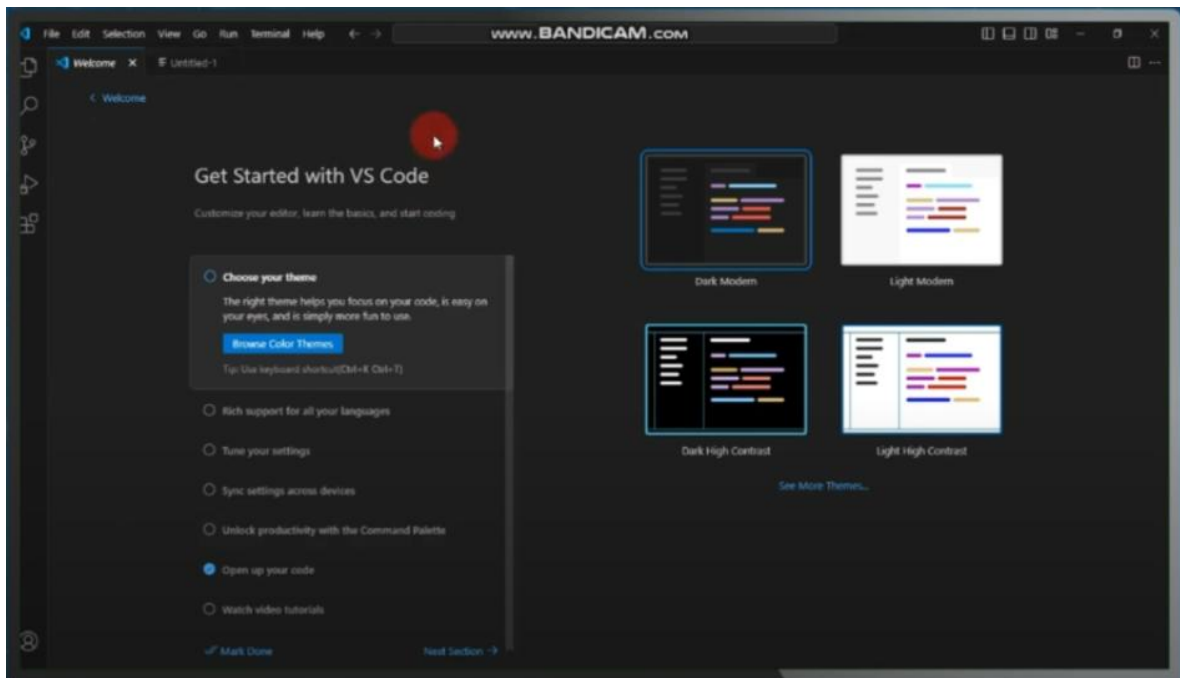
  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

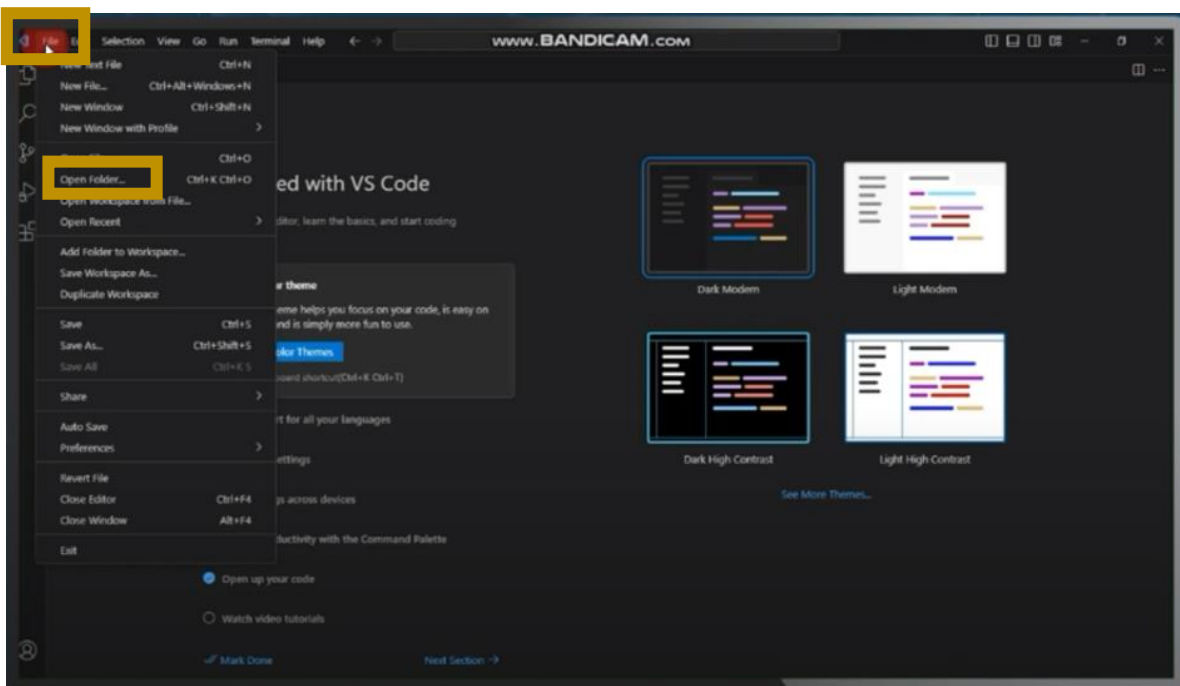
  cd reactjs
  npm start

Happy hacking!
```

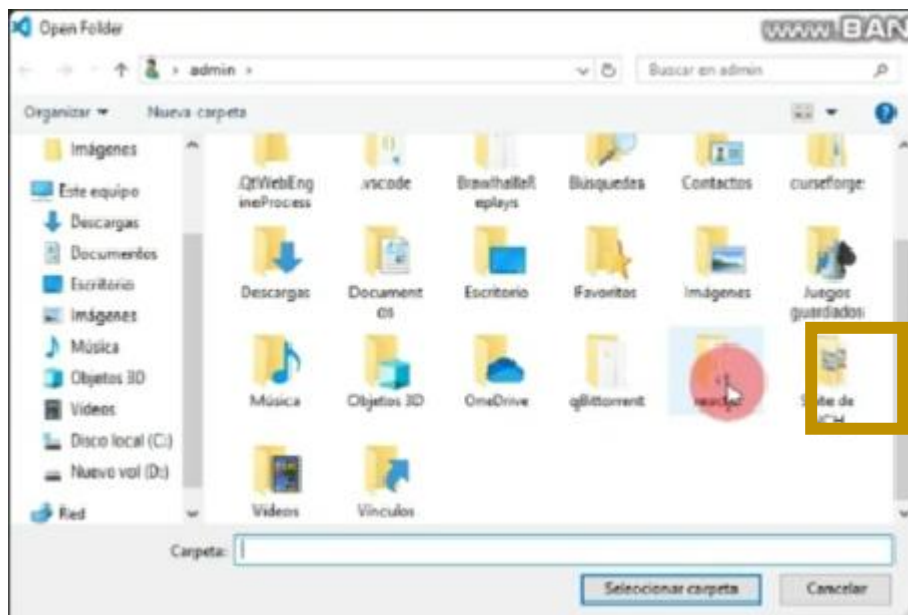
Ahora procede a abrir Visual Studio Code



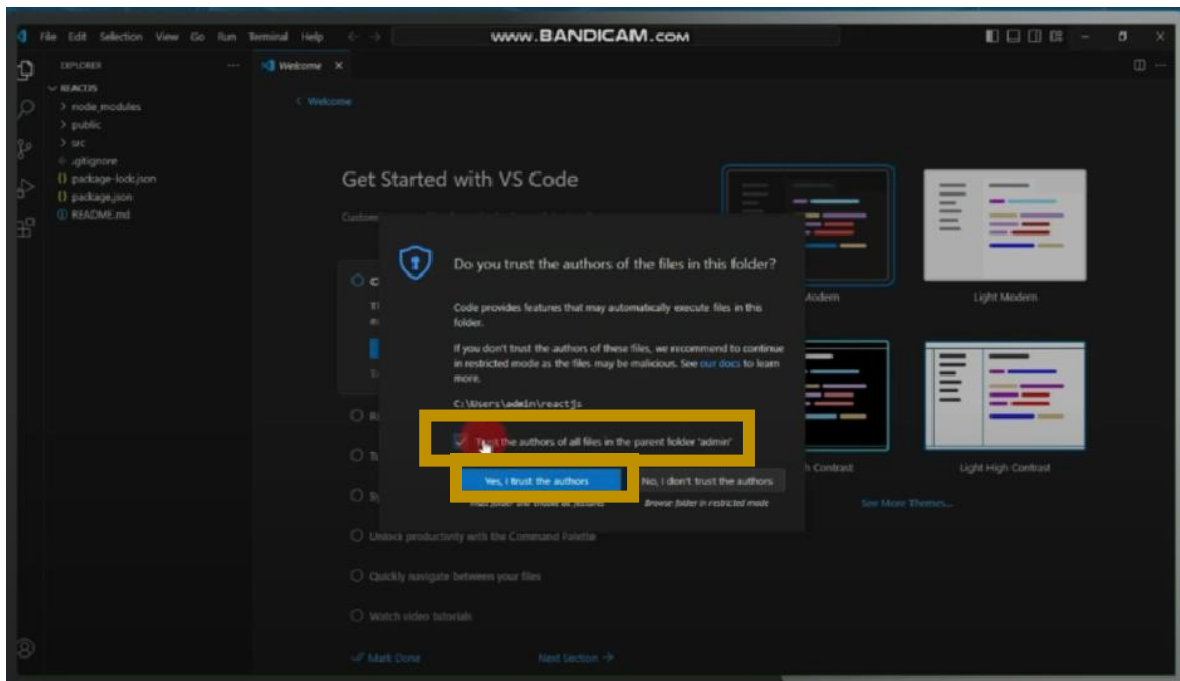
Dale click es “File” y “Open Folder”



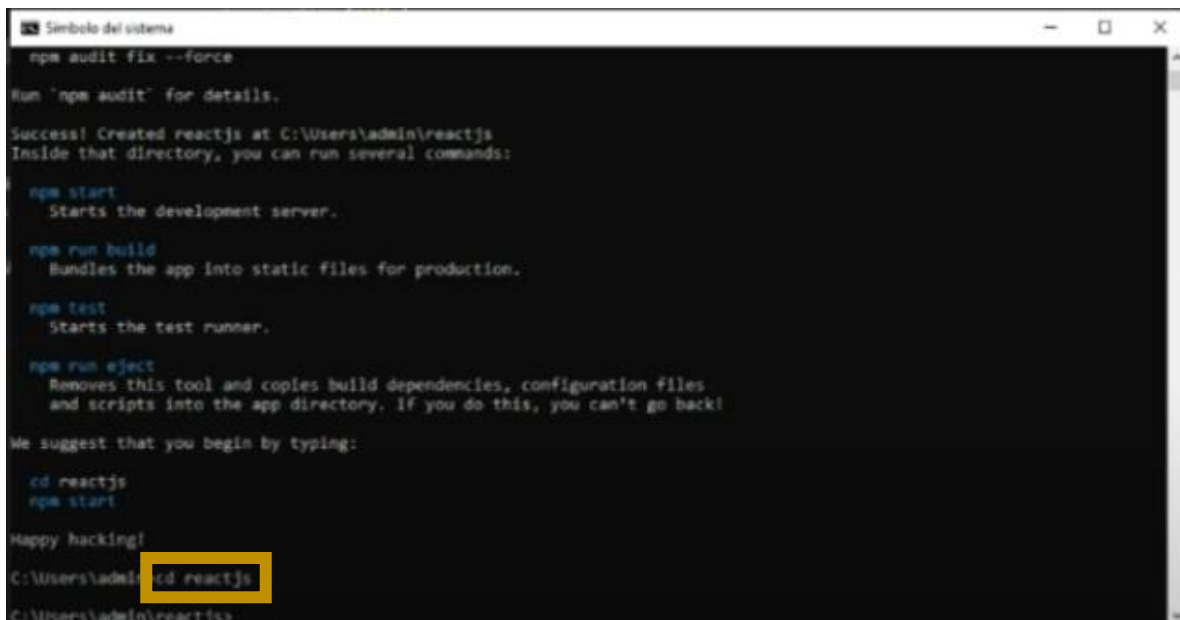
En las carpetas del admin busca la de “Reactjs” selecciona



Click en confiar en esta fuente y dile que si



Volveremos a la consola a escribir “cd reactjs” y enter



Luego escribe “npm start”, enter y darle permitir acceso

```
npm
Run "npm audit" for details.
Success! Created reactjs at C:\Users\admin\reactjs
Inside that directory, you can run several commands:

  npm start
    Starts the development server.

  npm run build
    Bundles the app into static files for production.

  npm test
    Starts the test runner.

  npm run eject
    Removes this tool and copies build dependencies, configuration files
    and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

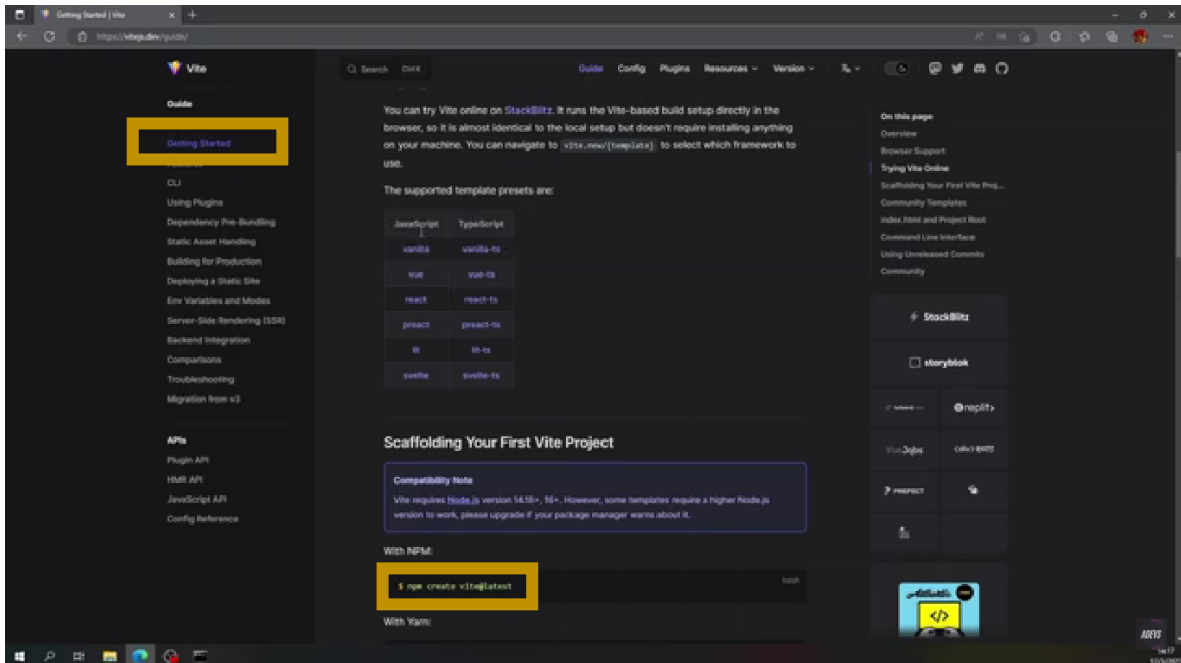
  cd reactjs
  npm start

Happy hacking!
C:\Users\admin>cd reactjs
C:\Users\admin\reactjs>npm start
```

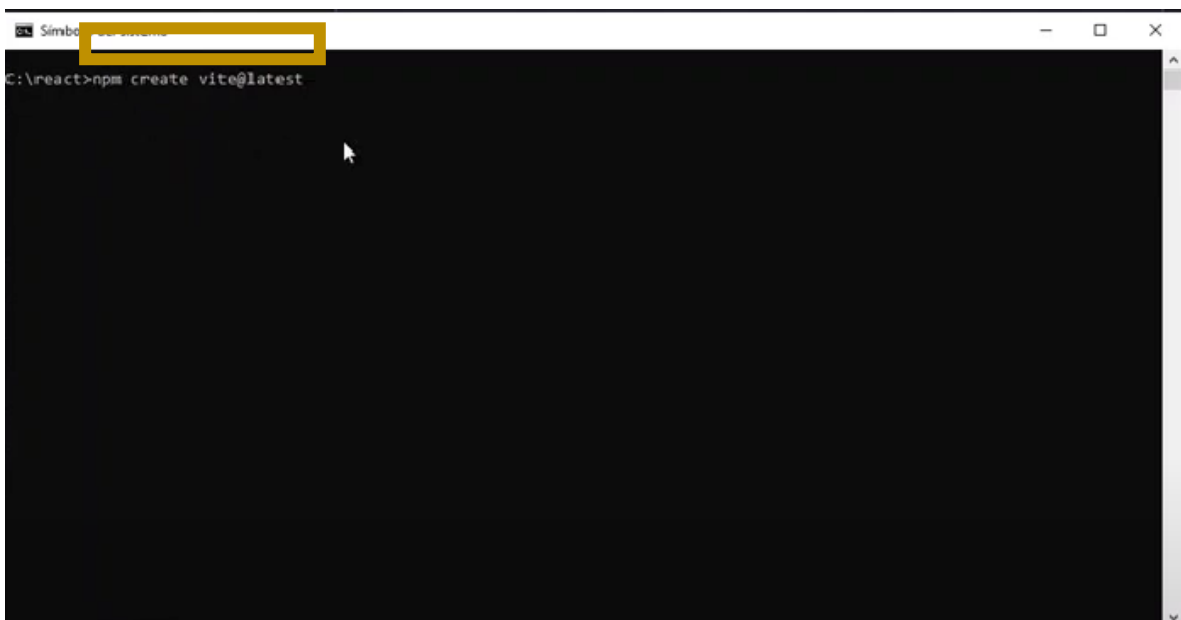


Vite

En la página oficial vete a “Getting Started” y al bajar hasta este punto copia el siguiente código.



Vamos por nuestra consola de nodejs donde pegaras lo anterior



Ponemos el nombre de nuestro proyecto y seleccionamos a React

```
C:\Windows\system32\cmd.exe

C:\react>npm create vite@latest
✓ Project name: ... test-youtube
? Select a framework: » - Use arrow-keys. Return to submit.
  Vanilla
  Vue
  React
  > Preact
  Svelte
  Others
```

Se selecciona JavaScript

```
C:\Windows\system32\cmd.exe

C:\react>npm create vite@latest
✓ Project name: ... test-youtube
✓ Select a framework: » React
? Select a language: » - Use arrow-keys. Return to submit.
  JavaScript
  > JavaScript + SWC
  TypeScript + SWC
```

La consola te recomendará los siguientes comandos que debes poner

```
Simbolo del sistema
C:\react>npm create vite@latest
✓ Project name: ... test-youtube
✓ Select a framework: = React
✓ Select a variant: = JavaScript
Scaffolding project in C:\react\test-youtube...
Done. Now run:

  cd test-youtube
  npm install
  npm run dev

C:\react>cd test
```

Utiliza “cd Storysphere” para entrar en la carpeta

```
Simbolo del sistema
Scaffolding project in C:\react\test-youtube...
Done. Now run:

  cd test-youtube
  npm install
  npm run dev

C:\react>cd test-youtube
C:\react\test-youtube>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: FA82-C8B7

Directorio de C:\react\test-youtube
12/03/2023  14:19    <DIR>      .
12/03/2023  14:19    <DIR>      ..
12/03/2023  14:06             253 .gitignore
12/03/2023  14:06             361 index.html
12/03/2023  14:19             416 package.json
12/03/2023  14:19    <DIR>      public
12/03/2023  14:19    <DIR>      src
12/03/2023  14:06             163 vite.config.js
                4 archivos             1.193 bytes
                4 dirs 330.469.781.504 bytes libres
```

Para abrir la carpeta creada pondremos “code .”

```
Simbolo del sistema
Done. Now run:
  cd test-youtube
  npm install
  npm run dev

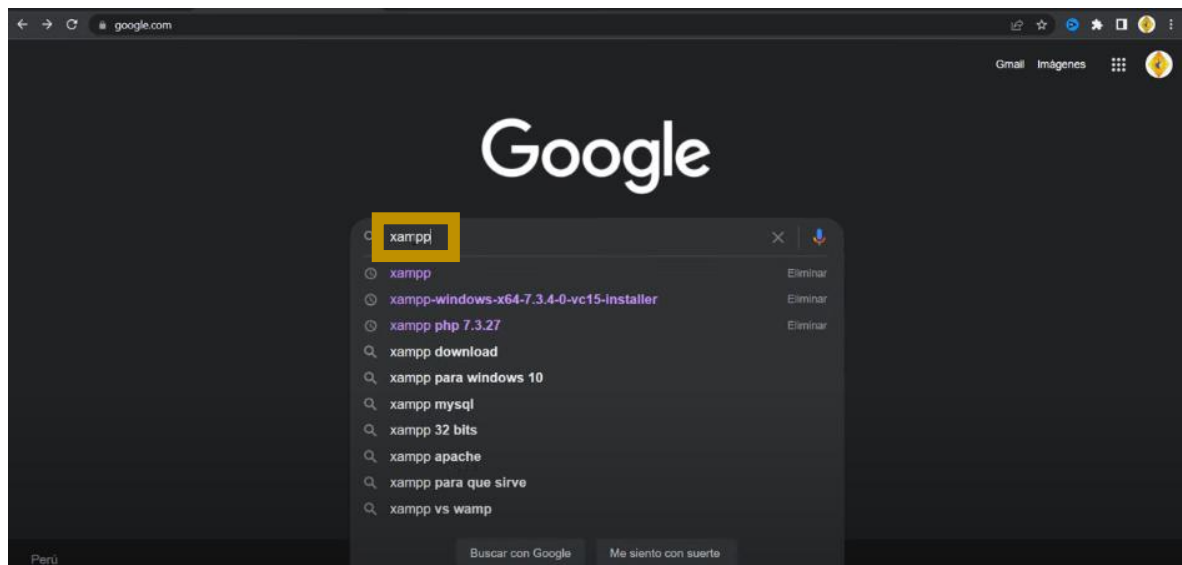
C:\react>cd test-youtube
C:\react\test-youtube>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: FA82-C8B7

Directorio de C:\react\test-youtube
12/03/2023  14:19  <DIR>      .
12/03/2023  14:19  <DIR>      ..
12/03/2023  14:06                253 .gitignore
12/03/2023  14:06                361 index.html
12/03/2023  14:19                416 package.json
12/03/2023  14:19  <DIR>      public
12/03/2023  14:19  <DIR>      src
12/03/2023  14:06                163 vite.config.js
                  4 archivos          1.193 bytes
                  4 dirs  330.469.781.504 bytes libres

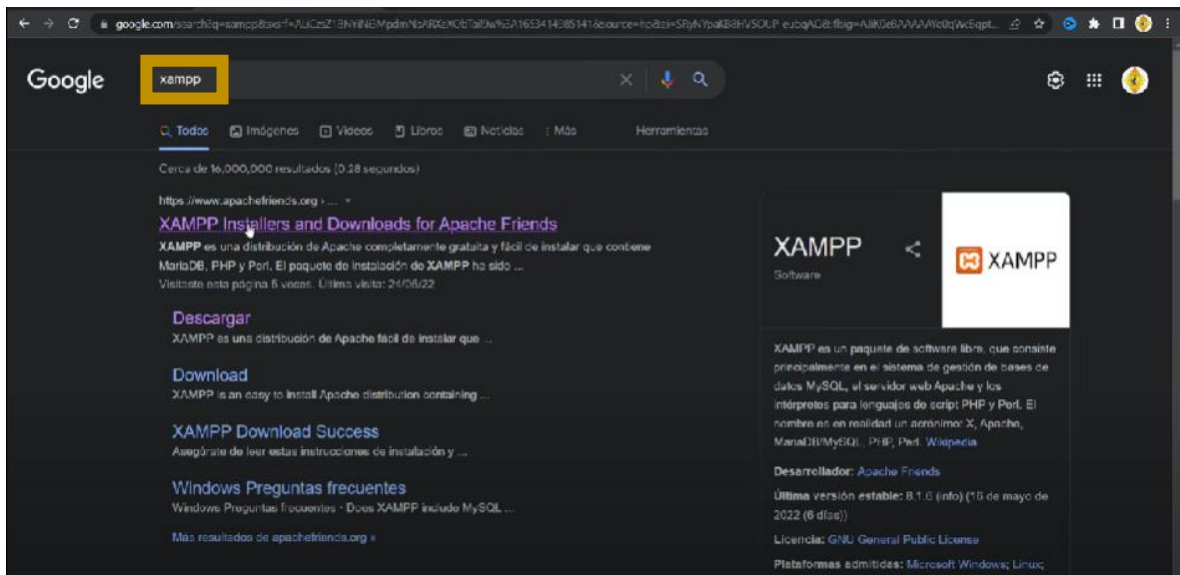
C:\react\test-youtube>code .
C:\react\test-youtube>
```

Servidor apache

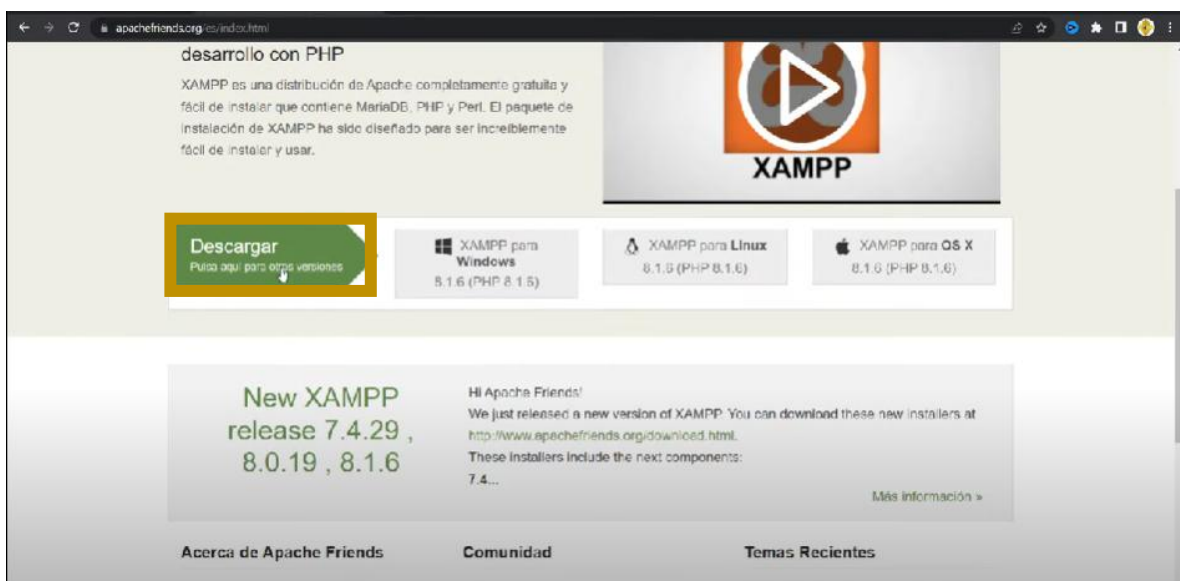
En este caso te enseñaremos a instalar xampp; ingresamos en nuestro navegador favorito el nombre “xampp”.



Irte a la página oficial de xampp

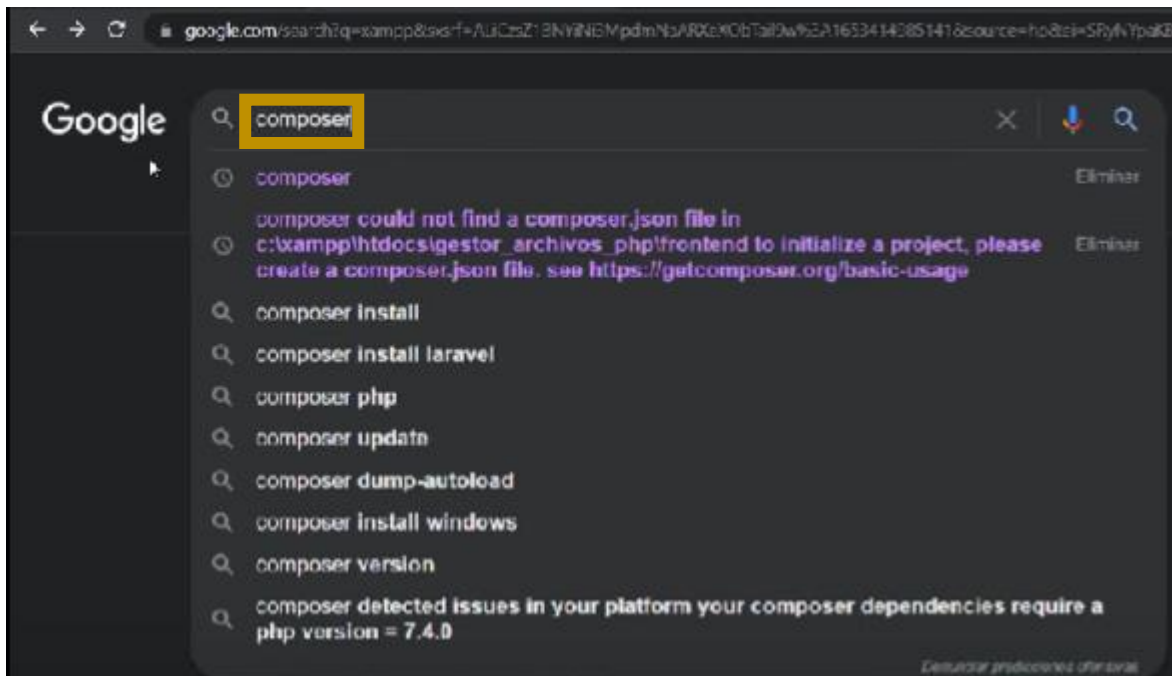


Ya en la pagina le daremos en descargar y se descargara la ultima versión por predeterminado

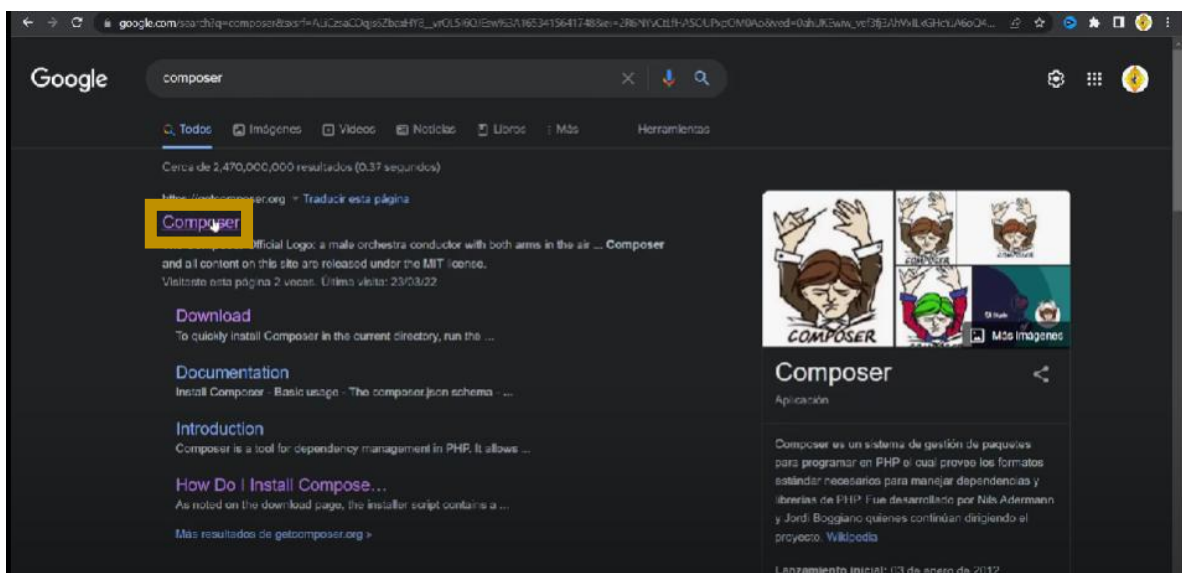


Composer

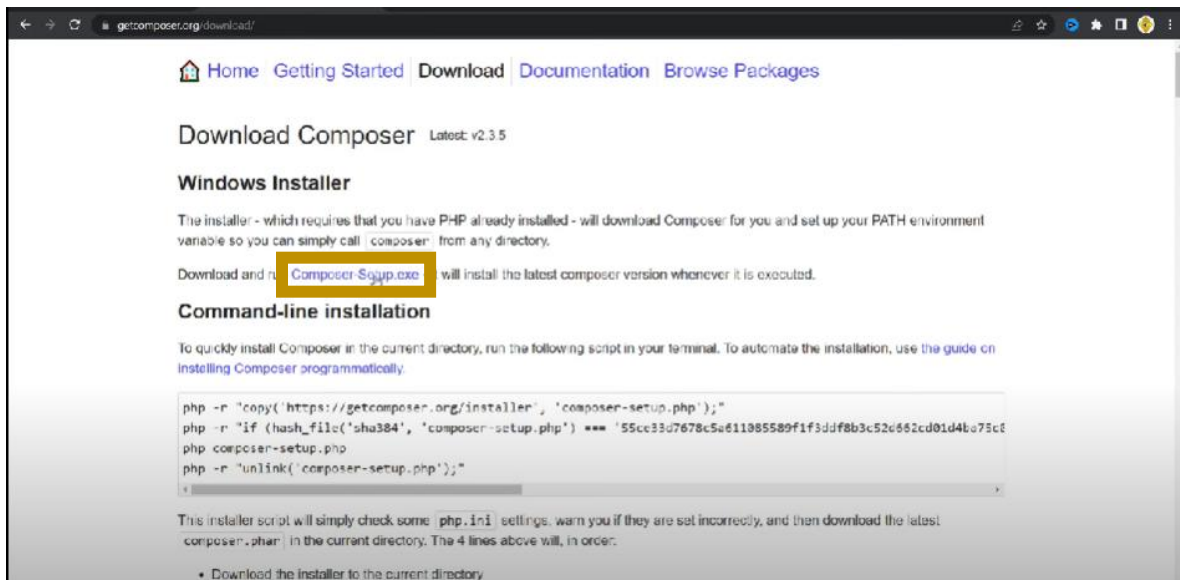
Colocar en nuestro navegador web favorito el nombre “composer”



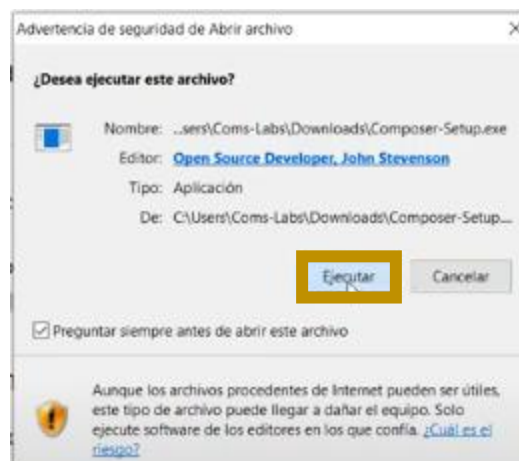
Irte a la pagina oficial de composer



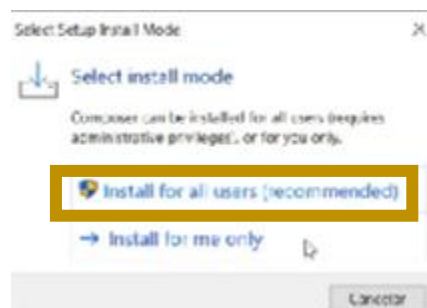
En la sección de Download vamos a darle click al siguiente link



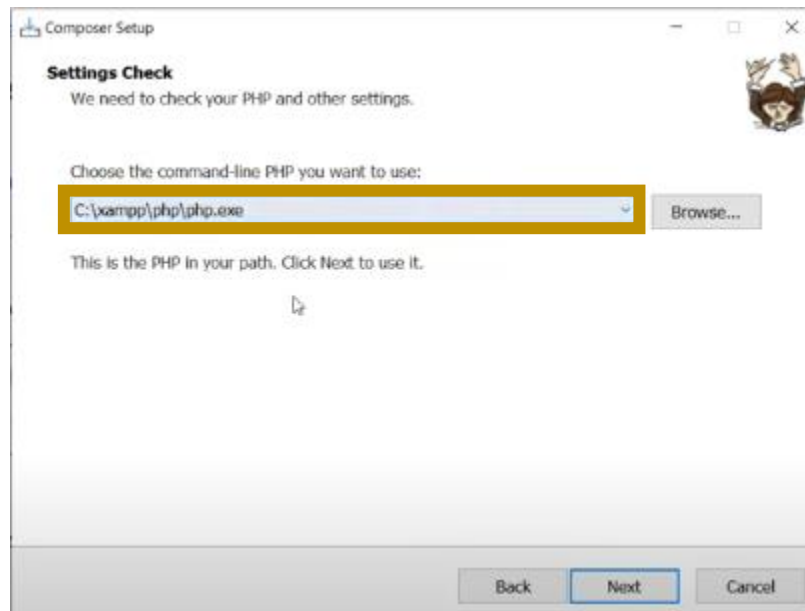
Abre el instalador y se le da click en ejecutar



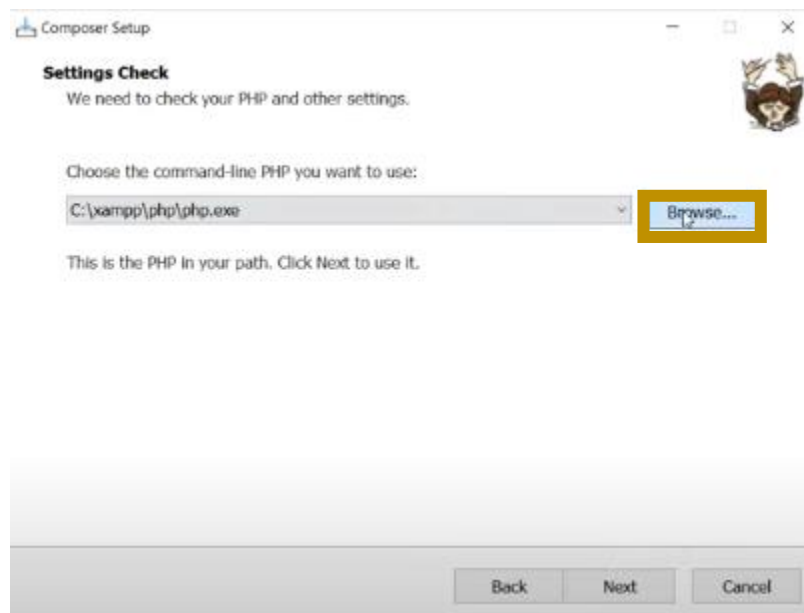
Escoge la instalación recomendada



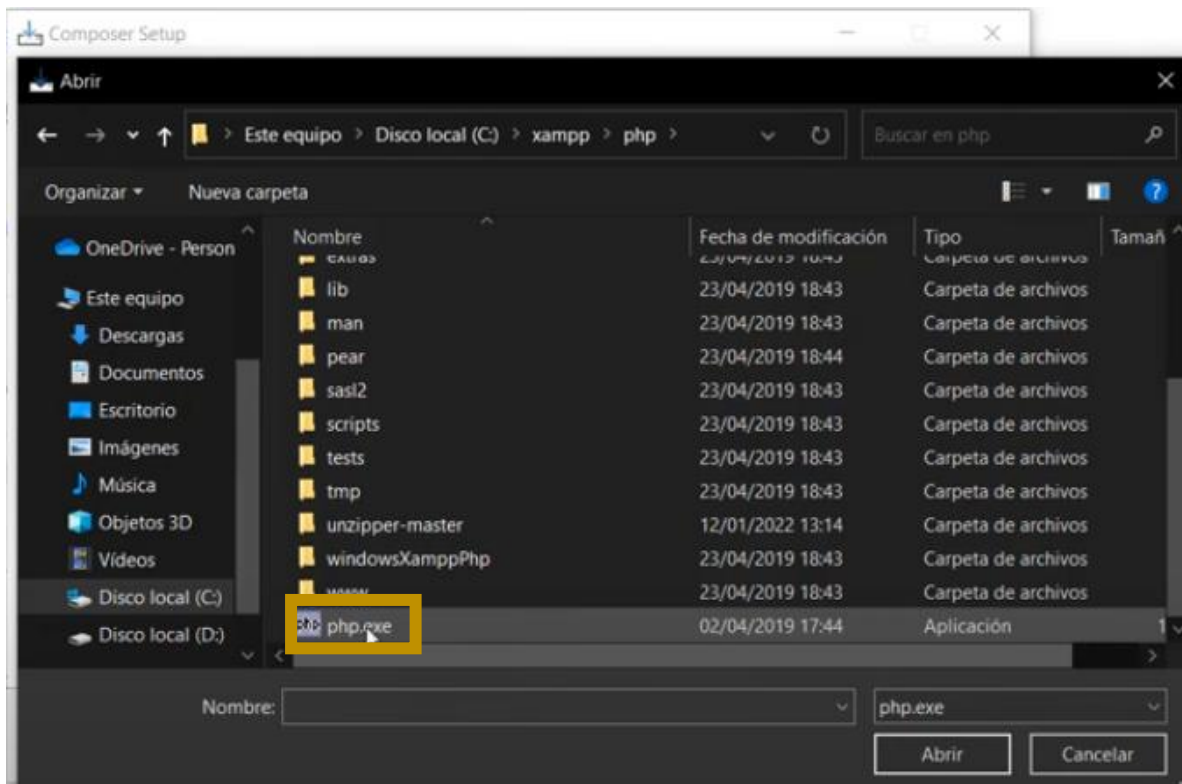
En la siguiente parte se le dará next y te saldrá el link donde se instalará, debería se en el servidor apache que tengas disponible en tu computadora.



En el caso de no salirte el link predeterminado de tu servidor apache instalado en el computador dale en “Browse”



Te vas a disco local C: y buscas la carpeta de xampp



Después de darle abrir daremos click en next y si aparece esta pantalla no seleccionaremos nada y de igual forma le daremos next.

Proxy Settings

Choose if you need to use a proxy.



☐ Use a proxy server to connect to internet

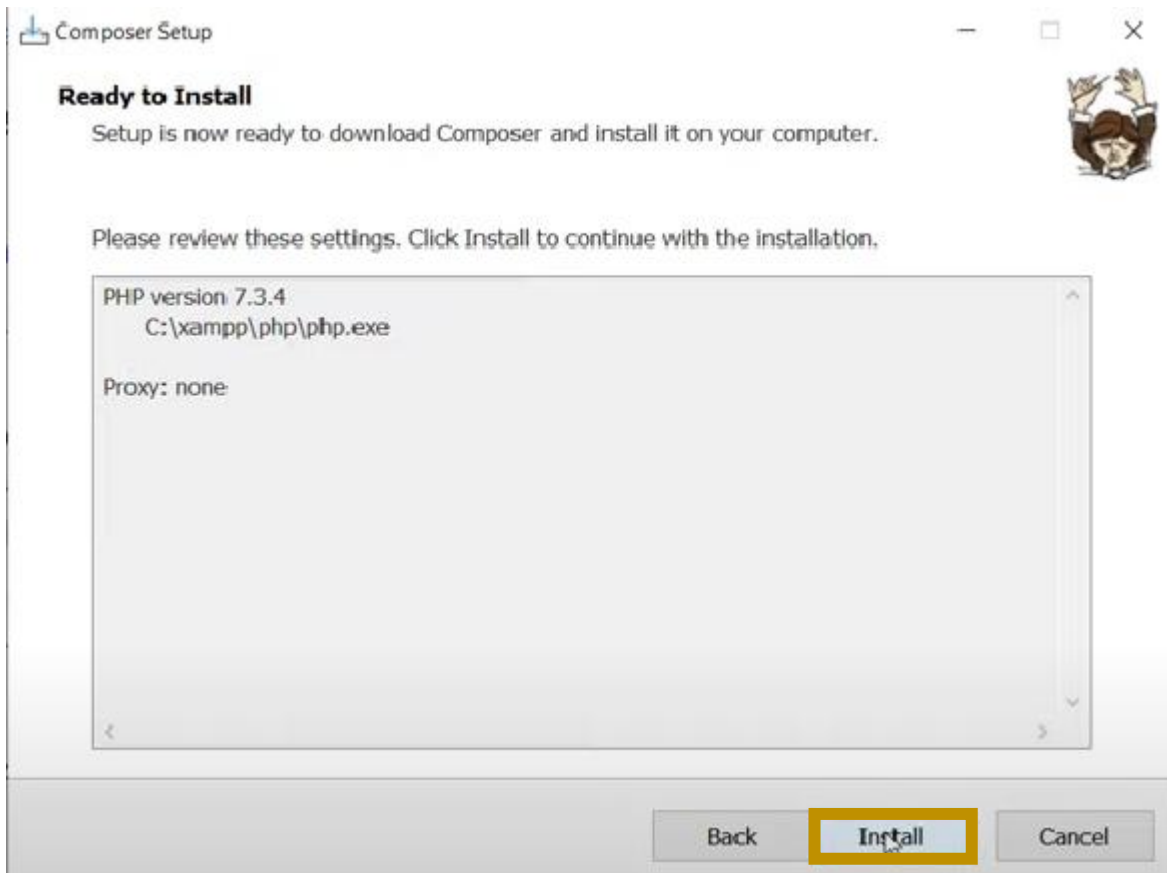
Enter proxy url

Back

Next

Cancel

Para seguir daremos install



Dos veces seguidas next y finish para terminar la instalación

Arquitectura del sistema

Estructura general del software

El sistema está diseñado con una arquitectura cliente-servidor, donde el frontend, el backend y la base de datos trabajan de manera interconectada para proporcionar una experiencia fluida a los usuarios.

Frontend – React.js con Vite

- Se encarga de la interacción con el usuario, proporcionando una interfaz gráfica dinámica y responsiva.
- Utiliza React.js para la gestión de componentes reutilizables.

- Se apoya en Vite para una compilación rápida y eficiente.
- Usa bibliotecas como React Router para la navegación, Axios para consumir la API, y Framer Motion para animaciones.

Backend – Laravel

- Implementa una API RESTful en PHP con Laravel para manejar las solicitudes del frontend.
- Gestiona la autenticación de usuarios, validaciones y la lógica del negocio.
- Se encarga de procesar datos y devolver respuestas en formato JSON al frontend.

Base de datos – MYSQL

- Almacena y gestiona la información del sistema de forma estructurada.
- Utiliza MYSQL, un sistema de bases de datos relacionales (RDBMS) con alta escalabilidad.
- Laravel interactúa con la base de datos a través de Eloquent ORM, facilitando la manipulación de datos con PHP.

Comunicación entre módulos

- Frontend ↔ Backend: Se comunican a través de peticiones HTTP utilizando Axios en React y controladores en Laravel.
- Backend ↔ Base de datos: Laravel usa Eloquent ORM y consultas SQL para interactuar con SQL Server.

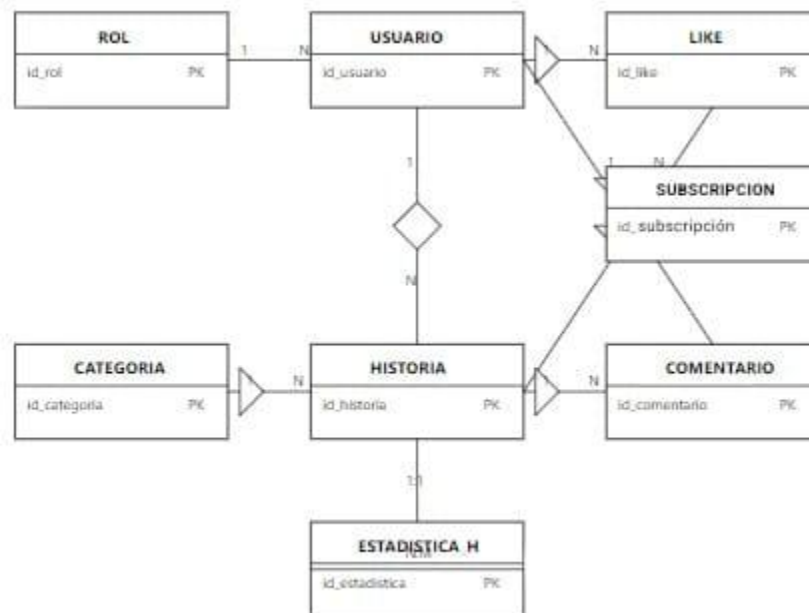
Desarrollo y estructura del código

Organización del código (Carpetas y archivos)

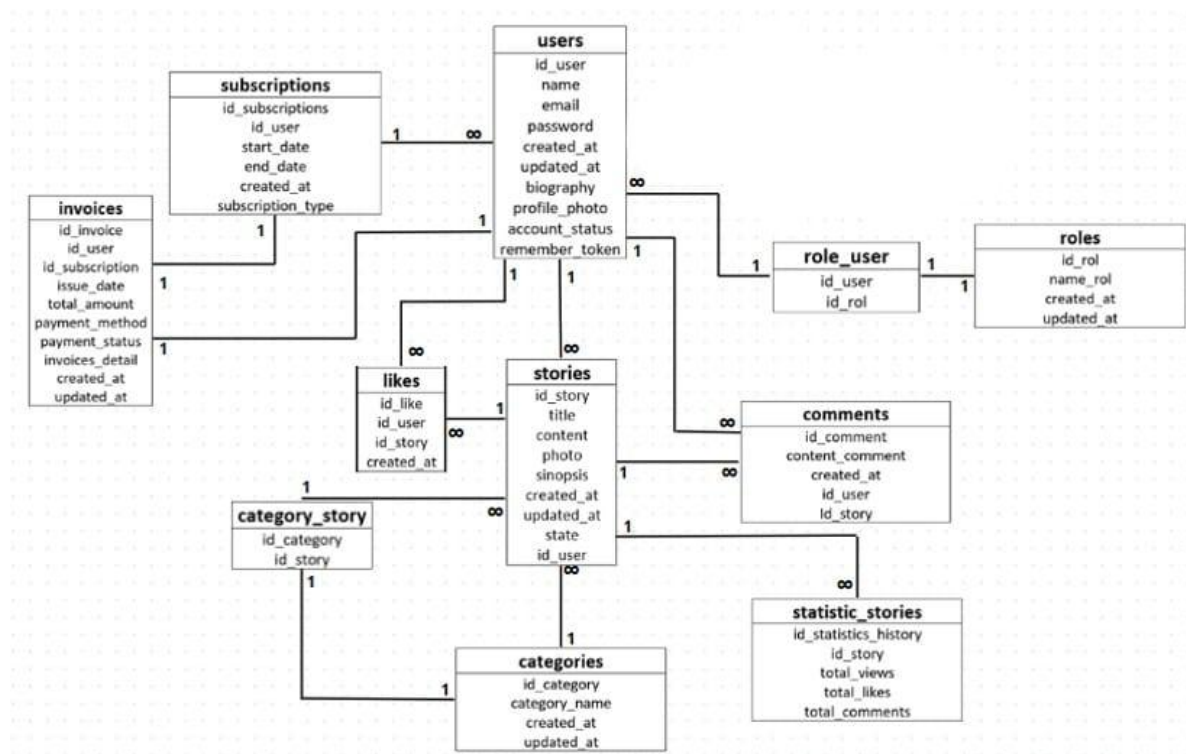
Base de datos

Modelo de datos y diagramas entidad-relación

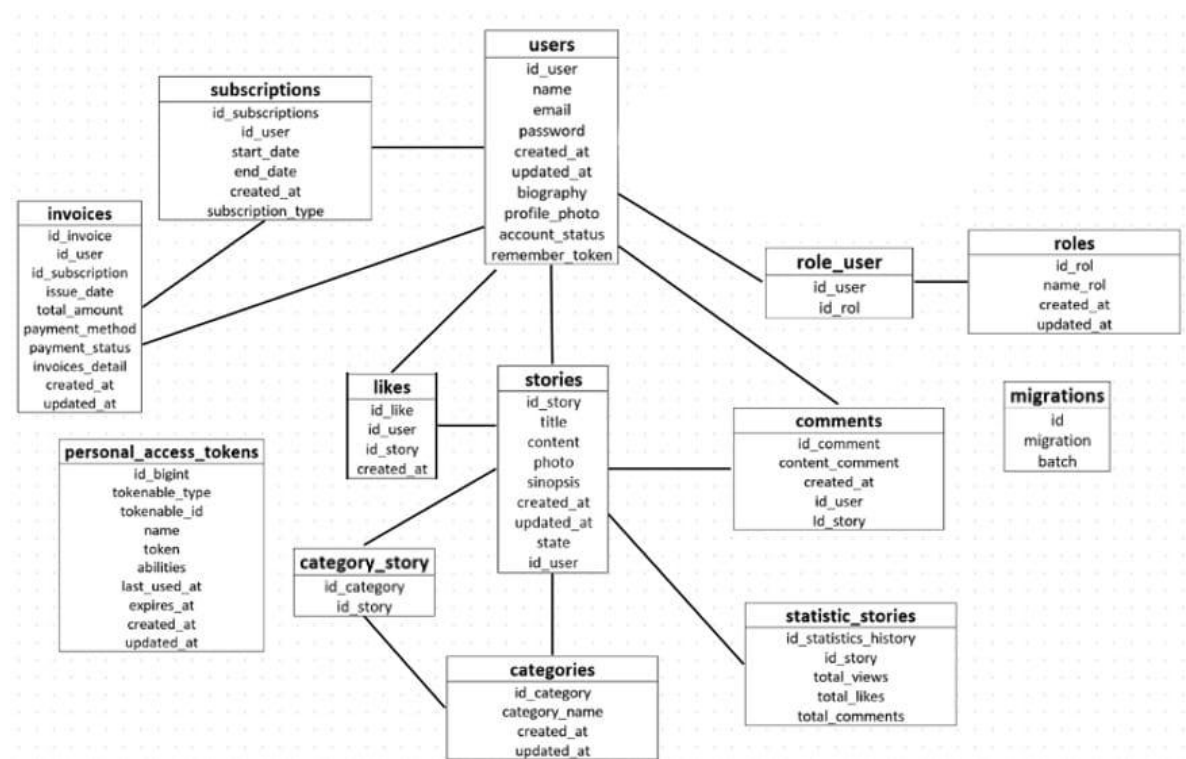
Modelo de datos



Modelo entidad - relación



Tablas principales y relaciones



Procedimientos almacenados y consultas SQL relevantes

-- Crear tabla de usuarios

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL,  
    role ENUM('user', 'admin') DEFAULT 'user',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP  
);
```

-- Procedimiento para registrar un usuario

```
DELIMITER //  
  
CREATE PROCEDURE RegisterUser(IN p_name VARCHAR(255), IN p_email  
VARCHAR(255), IN p_password VARCHAR(255))  
  
BEGIN  
  
    INSERT INTO users (name, email, password) VALUES (p_name, p_email,  
p_password);  
  
END //  
  
DELIMITER ;
```

-- Procedimiento para obtener todos los usuarios


```
DELIMITER //
```

```
CREATE PROCEDURE GetAllUsers()
```

```
BEGIN
```

```
    SELECT id, name, email, role FROM users;
```

```
END //
```

```
DELIMITER ;
```

```
-- Procedimiento para eliminar un usuario
```

```
DELIMITER //
```

```
CREATE PROCEDURE DeleteUser(IN p_id INT)
```

```
BEGIN
```

```
    DELETE FROM users WHERE id = p_id;
```

```
END //
```

```
DELIMITER ;
```

```
-- Crear tabla de historias
```

```
CREATE TABLE stories (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    user_id INT,
```

```
    title VARCHAR(255) NOT NULL,
```

```
    content TEXT NOT NULL,
```

```
    status ENUM('public', 'private') DEFAULT 'public',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
CURRENT_TIMESTAMP,  
  
FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

-- Procedimiento para obtener todas las historias

```
DELIMITER //
```

```
CREATE PROCEDURE GetAllStories()
```

```
BEGIN
```

```
SELECT s.id, s.title, s.content, u.name AS author, s.status
```

```
FROM stories s
```

```
JOIN users u ON s.user_id = u.id;
```

```
END //
```

```
DELIMITER ;
```

-- Procedimiento para crear una historia

```
DELIMITER //
```

```
CREATE PROCEDURE CreateStory(IN p_user_id INT, IN p_title VARCHAR(255), IN  
p_content TEXT)
```

```
BEGIN
```

```
INSERT INTO stories (user_id, title, content) VALUES (p_user_id, p_title, p_content);
```

```
END //
```

```
DELIMITER ;
```

-- Crear tabla de comentarios

```
CREATE TABLE comments (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    story_id INT,  
    user_id INT,  
    comment TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (story_id) REFERENCES stories(id),  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

-- Procedimiento para obtener comentarios de una historia

```
DELIMITER //  
  
CREATE PROCEDURE GetCommentsByStory(IN p_story_id INT)  
  
BEGIN  
  
    SELECT c.id, c.comment, u.name AS author, c.created_at  
  
    FROM comments c  
  
    JOIN users u ON c.user_id = u.id  
  
    WHERE c.story_id = p_story_id;  
  
END //  
  
DELIMITER ;
```

-- Procedimiento para agregar un comentario a una historia

```
DELIMITER //
```

```
CREATE PROCEDURE AddComment(IN p_story_id INT, IN p_user_id INT, IN
p_comment TEXT)

BEGIN

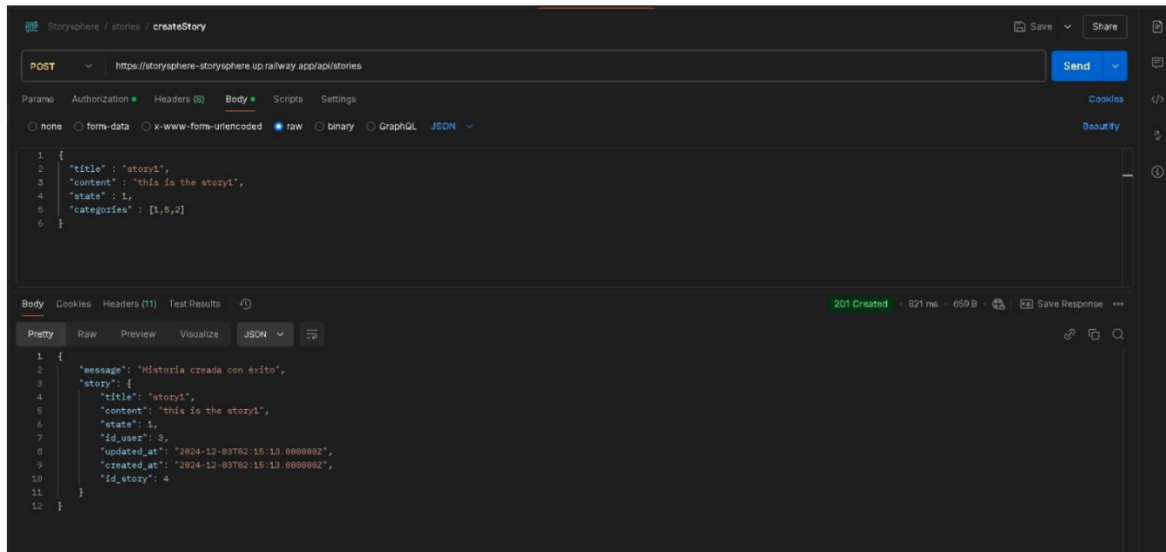
    INSERT INTO comments (story_id, user_id, comment) VALUES (p_story_id,
p_user_id, p_comment);

END //

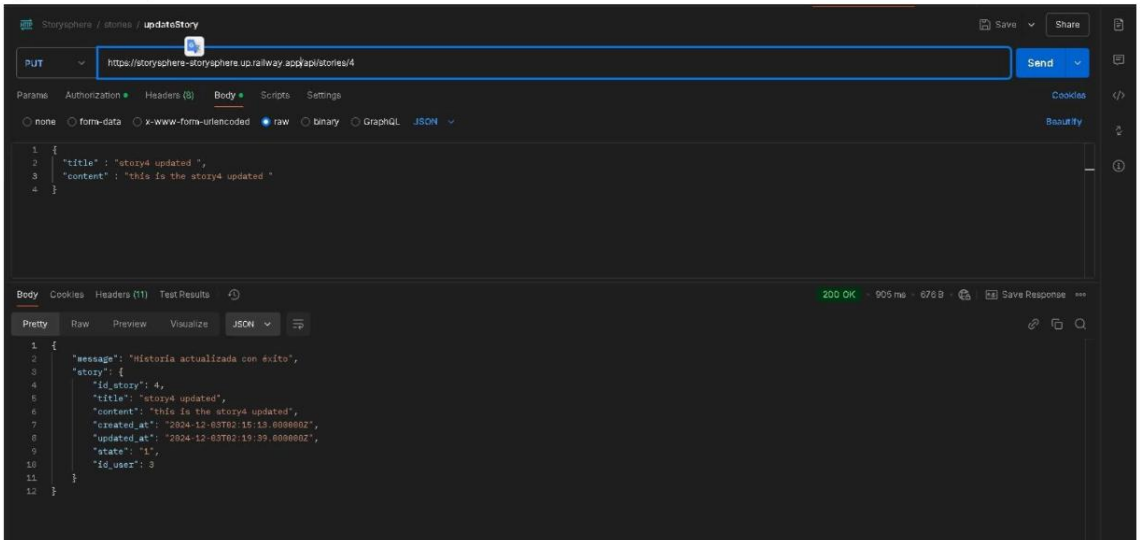
DELIMITER ;
```

Ejemplos de solicitudes y respuestas

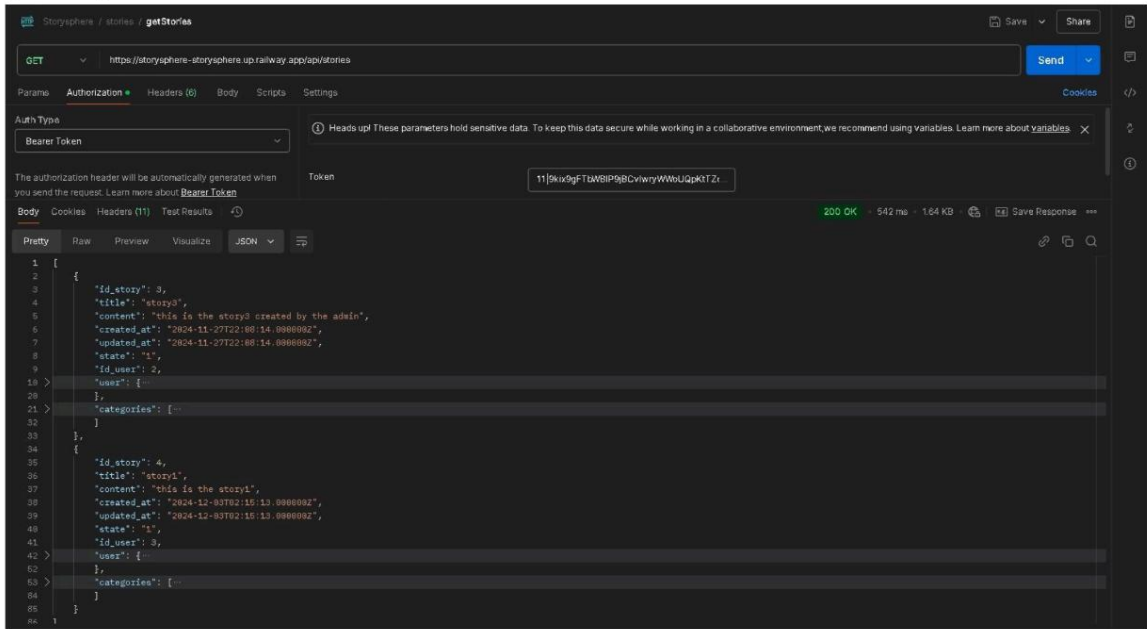
create story



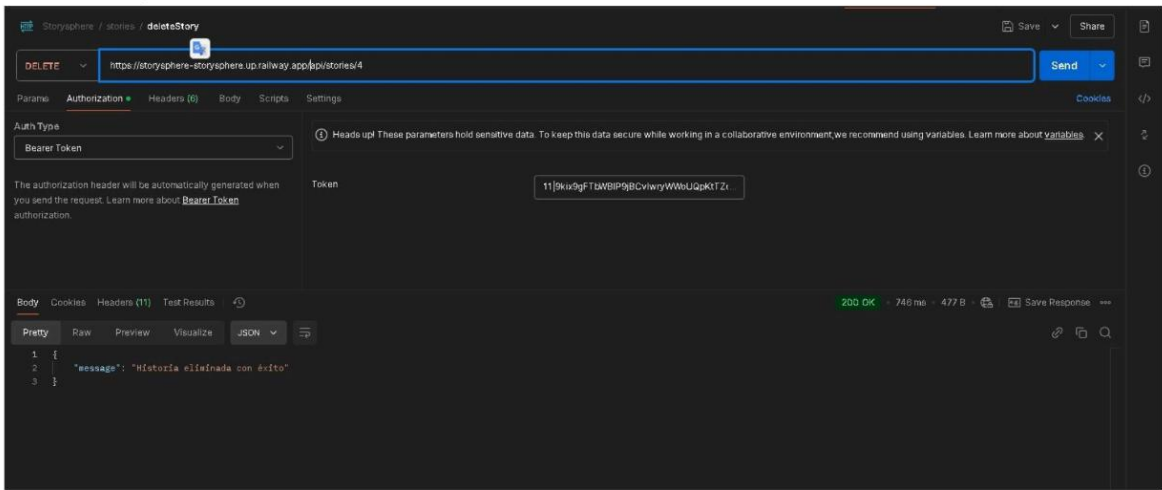
updateStory



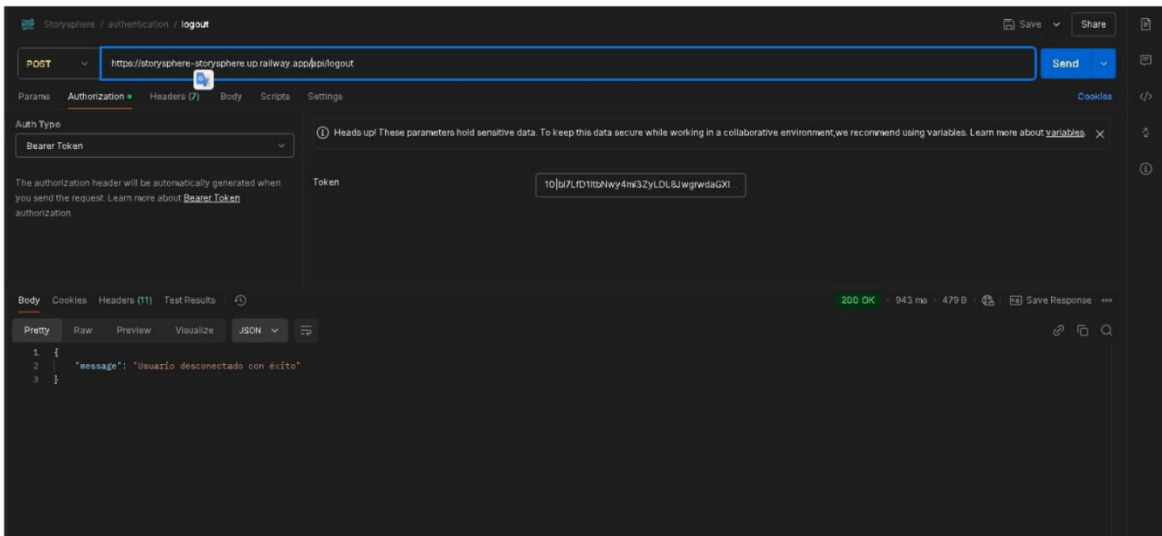
getStory



deleteStory



user



Login

The screenshot shows a REST client interface for a POST request to `https://storysphere-storysphere.up.railway.app/api/login`. The request body is a JSON object with email and password. The response is a 200 OK status with a detailed JSON payload containing user information and roles.

```
POST https://storysphere-storysphere.up.railway.app/api/login

{
  "email": "example1@gmail.com",
  "password": "example1234"
}
```

```
{
  "message": "Usuario autenticado con éxito",
  "user": {
    "id_user": 3,
    "name": "example1",
    "email": "example1@gmail.com",
    "created_at": null,
    "updated_at": null,
    "biography": "This is the example1",
    "profile_photo": null,
    "account_status": "active",
    "active": 1,
    "roles": [
      {
        "id_role": 1,
        "name_role": "user",
        "created_at": null,
        "updated_at": null,
        "pivot": {
          "id_user": 3,
          "id_role": 1
        }
      }
    ]
  }
}
```

Register

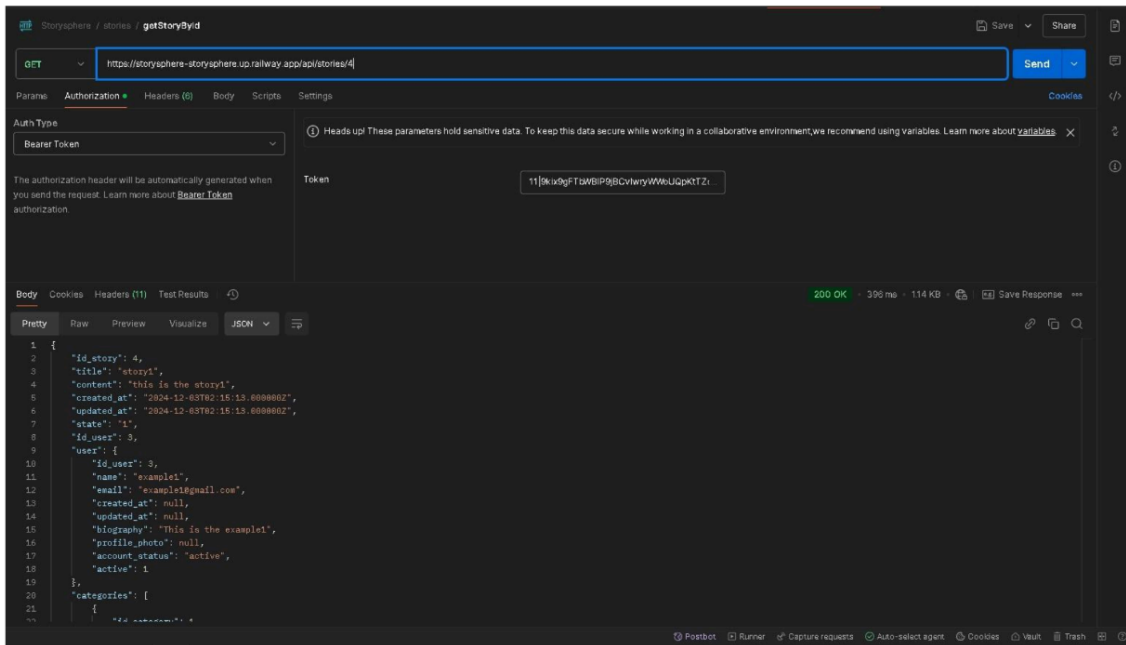
The screenshot shows a REST client interface for a POST request to `https://storysphere-storysphere.up.railway.app/api/register`. The request body is a JSON object with name, email, password, and biography. The response is a 201 Created status with a detailed JSON payload containing user information and roles.

```
POST https://storysphere-storysphere.up.railway.app/api/register

{
  "name": "example1",
  "email": "example1@gmail.com",
  "password": "example1234",
  "biography": "This is the example1"
}
```

```
{
  "message": "Usuario creado con éxito",
  "user": {
    "id_user": 3,
    "name": "example1",
    "email": "example1@gmail.com",
    "created_at": null,
    "updated_at": null,
    "biography": "This is the example1",
    "profile_photo": null,
    "account_status": "active",
    "active": 1,
    "roles": [
      {
        "id_role": 1,
        "name_role": "user",
        "created_at": null,
        "updated_at": null,
        "pivot": {
          "id_user": 3,
          "id_role": 1
        }
      }
    ]
  }
}
```

getByldStory



Bibliografía

Instalaciones

Visual Studio Code

https://www.youtube.com/watch?v=6pD7_rcFrj8

React apartir de Nodejs

<https://www.youtube.com/watch?v=EXiF5FPIKvY>

React + Vite

<https://www.youtube.com/watch?v=LxMCo8HZ6kM>

Xampp y Composer

<https://www.youtube.com/watch?v=yp04wvbAJP>

Código fuente

<https://github.com/yey10/storysphere>

Diagramas

<https://docs.google.com/document/d/1iwcntjE7IFCbHVmFEYtLLLGJyuM9Y3txELb1TJgAqpk/edit?usp=sharing>